

L'interprete Lisp di rsedit

Come funziona, dal testo al valore — manuale per lo sviluppatore

Indice

1	Introduzione	3
1.1	Come leggere questo manuale	3
1.2	Comportamento e difetti	3
1.3	Convenzioni	3
2	Panoramica	4
2.1	I file e i livelli	4
2.2	Le quattro strutture portanti	4
2.3	Il percorso di un'espressione	5
2.4	Primo esempio: una chiamata a una primitiva	6
2.5	Secondo esempio: una funzione ricorsiva in coda	6
2.6	Gli invarianti su cui si regge tutto	7
3	Il modello dei dati	8
3.1	LispExp, il tipo di tutto	8
3.2	Clonare non copia	8
3.3	Costruire e scorrere le liste	9
3.4	nil, t e la verità	9
3.5	L'obarray	10
3.6	Uguaglianza	10
3.7	Stampa	11
3.8	Liberare una lista lunga	12
3.9	Sintassi e dati: Form contro Cons	12
3.10	Le strutture di types.rs	14
4	Il lettore	15
4.1	L'interfaccia	15
4.2	Lo stato del Parser	15
4.3	Il lexer	15
4.4	Il parser	17
5	Gli ambienti	20
5.1	La struttura di Env	20
5.2	La ricerca	20
5.3	La scrittura	21
5.4	Come lega ciascuna forma	21
5.5	Lo scope è lessicale	22
5.6	Le proprietà dei simboli	22
5.7	Letture tipizzate ed elenchi di nomi	22
6	Il valutatore	24
6.1	Il trampolino	24
6.2	Il permesso di sospendersi	25
6.3	Lo smistamento per tipo	25

6.4	Le forme speciali	26
6.5	Chiamate di funzione e macro	30
6.6	La pila delle chiamate per i backtrace	34
7	Errori e uscite non locali	35
7.1	Un solo canale per tre cose diverse	35
7.2	Come si propaga	36
7.3	catch e throw	36
7.4	condition-case	37
7.5	unwind-protect	38
7.6	signal, eval-string-safe e gli errori dell'host	38
8	Fiber e yield	39
8.1	Il problema: lo stack di Rust non si può conservare	39
8.2	La rappresentazione	39
8.3	Dove si può sospendere	39
8.4	Lo srotolamento: chi scrive i frame	40
8.5	La ripresa: resume e resume_frames	41
8.6	L'esempio, frame per frame	42
9	Il carburante	43
9.1	Il contatore e il budget	43
9.2	Le operazioni	43
9.3	Dove si paga	44
9.4	Il residuo durante un comando	44
10	Thread e stato condiviso	45
11	Il contratto con l'host	46
11.1	Il trait <code>LispContext</code>	46
11.2	L'avvio: <code>bootstrap_vm</code>	47
11.3	Cosa l'interprete esporta	47
12	Le primitive di base	48
12.1	Organizzazione	48
12.2	Gli strumenti comuni	48
12.3	Semantiche da conoscere	49
12.4	Ricetta: aggiungere una primitiva di base	50
13	Differenze da Emacs Lisp	51
14	Estendere l'interprete	52
14.1	Una nuova forma speciale	52
14.2	Una nuova variante di <code>LispExp</code>	52
14.3	I test	52
15	Appendice	53
15.1	Glossario	53
15.2	Mantenere aggiornato questo manuale	53

1 Introduzione

Questo manuale spiega come funziona l'interprete Lisp che vive in `core/src/lisp/`. Segue un'espressione dal testo sorgente fino al valore e, a ogni tappa, descrive le strutture che attraversa, l'algoritmo che le trasforma e il motivo per cui l'algoritmo è fatto così. È scritto per chi deve modificare l'interprete, estenderlo o incorporarlo in un altro programma.

L'interprete è *agnostico rispetto al contesto*. Non conosce buffer, finestre, tasti o file: ogni suo tipo è generico su un parametro `T: LispContext`, e l'editor è soltanto uno dei `T` possibili. Il manuale compagno, *L'editor di rseedit* (`architettura-editor.typ`), descrive l'altro lato del confine. Il documento `lisp-fibers.typ` approfondisce la storia e le scelte di progetto delle fiber; qui se ne descrive il meccanismo.

Il confine è verificato, non solo dichiarato: il test `tests::layering_tests` fallisce se un qualsiasi file sotto `lisp/` nomina un tipo dell'editor. Tutto ciò che questo manuale dice vale quindi per qualunque host incorpori l'interprete, compreso il contesto vuoto `()` usato dai test.

1.1 Come leggere questo manuale

Il Sezione 2 dà il quadro d'insieme: i file, le quattro strutture portanti e il viaggio di un'espressione, con due esempi seguiti passo per passo. I capitoli dal Sezione 3 al Sezione 10 descrivono ciascun meccanismo nell'ordine in cui un'espressione lo incontra: i dati, il lettore, gli ambienti, il valutatore, gli errori, le fiber, il carburante, i thread. Gli ultimi capitoli sono di riferimento: il contratto con l'host, le primitive di base, le differenze da Emacs Lisp e le ricette per estendere l'interprete.

Gli estratti di codice sono presi dal sorgente e semplificati: commenti tolti, parametri generici abbreviati, rami di errore ripetitivi riassunti. Ogni riga di un estratto corrisponde a una riga del file indicato sopra di esso.

1.2 Comportamento e difetti

Questo manuale descrive il comportamento *attuale* del codice, anche dove quel comportamento è un difetto. I difetti sono descritti, con la loro riproduzione e una correzione proposta, in un documento separato, il *Rapporto sui problemi del codice* (`rapporto-problemi.typ`). Dove il manuale descrive un punto che il rapporto considera un difetto, un rimando come (`→ INT-1`) indica la scheda corrispondente. Quando il difetto viene corretto, si aggiorna la descrizione qui e si toglie la scheda là.

1.3 Convenzioni

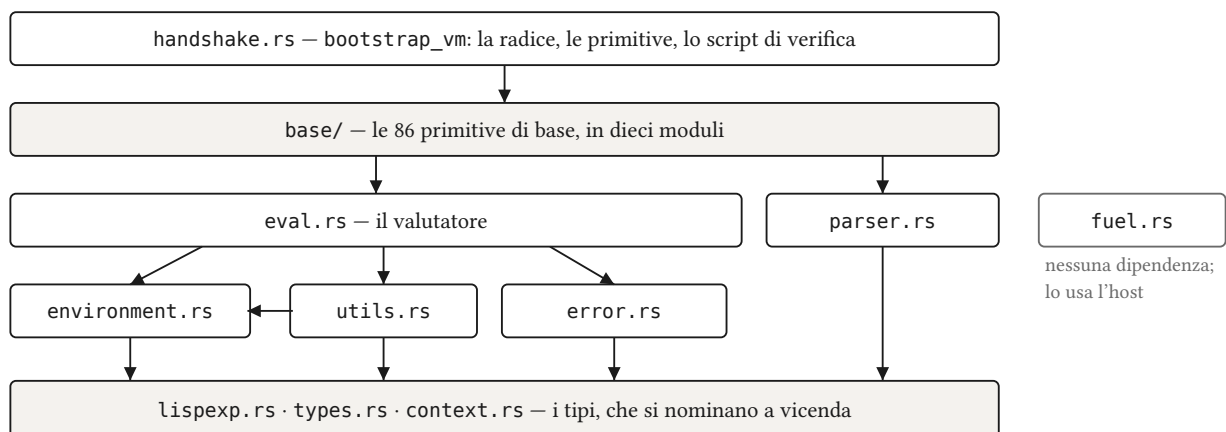
- I nomi in `monospazio` sono identificatori del codice: tipi e funzioni Rust (`eval_step`), primitive e forme Lisp (`condition-case`), file (`eval.rs`). I percorsi sono relativi a `core/src/lisp/`.
- «Forma» indica sempre un pezzo di *codice* «valore» o «dato» un risultato. La distinzione ha una controparte precisa nei tipi (Sezione 3.9).
- Nei diagrammi le caselle grigie sono dati, quelle bianche sono funzioni o passi, quelle azzurre appartengono all'host, quelle rosate sono errori.

2 Panoramica

2.1 I file e i livelli

File	Contenuto
<code>mod.rs</code>	Dichiarazione dei moduli e superficie esportata verso l'editor.
<code>context.rs</code>	Il trait <code>LispContext</code> : il contratto con l'host (Sezione 11).
<code>lispexp.rs</code>	<code>LispExp</code> , il tipo di ogni valore e di ogni forma; l'obarray dei simboli; uguaglianza e stampa.
<code>types.rs</code>	<code>ConsCell</code> , <code>Lambda</code> , <code>SharedAtom</code> , <code>Frame</code> , <code>FiberState</code> , <code>SharedFiber</code> , <code>LispPrimitive</code> .
<code>parser.rs</code>	Il lettore: lexer a stati e parser a discesa ricorsiva (Sezione 4); <code>form_to_data</code> .
<code>environment.rs</code>	<code>Env</code> : variabili, funzioni, macro e proprietà dei simboli (Sezione 5).
<code>eval.rs</code>	Il valutatore: trampolino, forme speciali, chiamate, sospensioni (Sezione 6).
<code>error.rs</code>	<code>EvalError</code> : errori e uscite non locali (Sezione 7).
<code>utils.rs</code>	Arità, grammatica dei parametri, legame degli argomenti, conversione dati → forme, mappatura errori → condizioni.
<code>fuel.rs</code>	Il meccanismo del carburante (Sezione 9). Non dipende da nulla e il valutatore non lo usa direttamente: serve all'host per implementare <code>consume_fuel</code> .
<code>handshake.rs</code>	<code>bootstrap_vm</code> : crea l'ambiente radice, installa le primitive e verifica che l'interprete funzioni.
<code>base/</code>	Le 86 primitive indipendenti dall'host, in dieci moduli (Sezione 12).
<code>tests/</code>	I test dell'interprete, eseguiti con il contesto <code>()</code> .

I file formano livelli: ognuno usa quelli sotto di sé e mai quelli sopra. La figura è ricavata dalle righe `use` dei sorgenti; una freccia significa «usa». Le dipendenze che scavalcano un livello non sono disegnate: `handshake.rs` , per esempio, chiama direttamente anche `eval` e `Parser` .



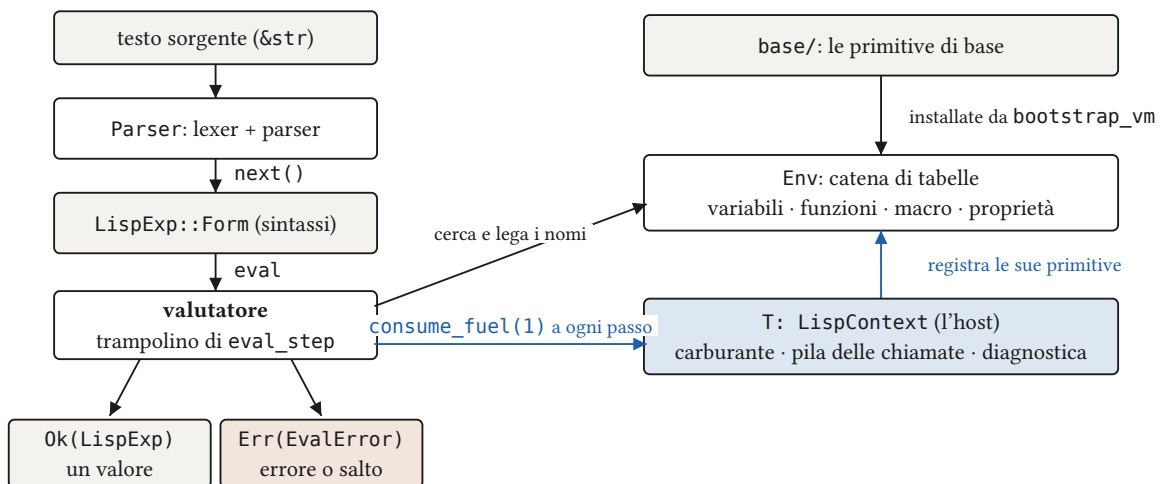
2.2 Le quattro strutture portanti

Quasi tutto l'interprete è il gioco di quattro tipi. Conviene conoscerli prima di leggere qualunque altra cosa.

Tipo	Che cos'è	Chi lo crea e chi lo consuma
<code>LispExp<T></code>	Ogni valore e ogni pezzo di codice: numeri, simboli, stringhe, liste, chiusure, primitive, atomi, fiber (Sezione 3).	Lo crea il lettore (dal testo) o una primitiva (a tempo di esecuzione); lo consuma il valutatore.
<code>Env<T></code>	Una catena di tabelle che associa nomi a valori, in tre spazi dei nomi separati (Sezione 5).	Lo crea <code>bootstrap_vm</code> (la radice) e ogni chiamata, <code>let</code> , ciclo o fiber (un anello figlio); lo consultano il valutatore e le primitive.
<code>EvalError<T></code>	Il ramo <code>Err</code> di ogni valutazione: un errore, un <code>throw</code> o una sospensione (Sezione 7).	Lo producono le forme e le primitive; lo fermano <code>catch</code> , <code>condition-case</code> , <code>resume</code> , oppure arriva all'host.
<code>T: LispContext</code>	L'host: chi paga il carburante, tiene la pila delle chiamate e riceve la diagnostica (Sezione 11).	Lo fornisce l'host; il valutatore lo riceve per riferimento a ogni passo.

2.3 Il percorso di un'espressione

Un programma attraversa sempre le stesse tappe. La figura le mostra con il tipo che passa dall'una all'altra.



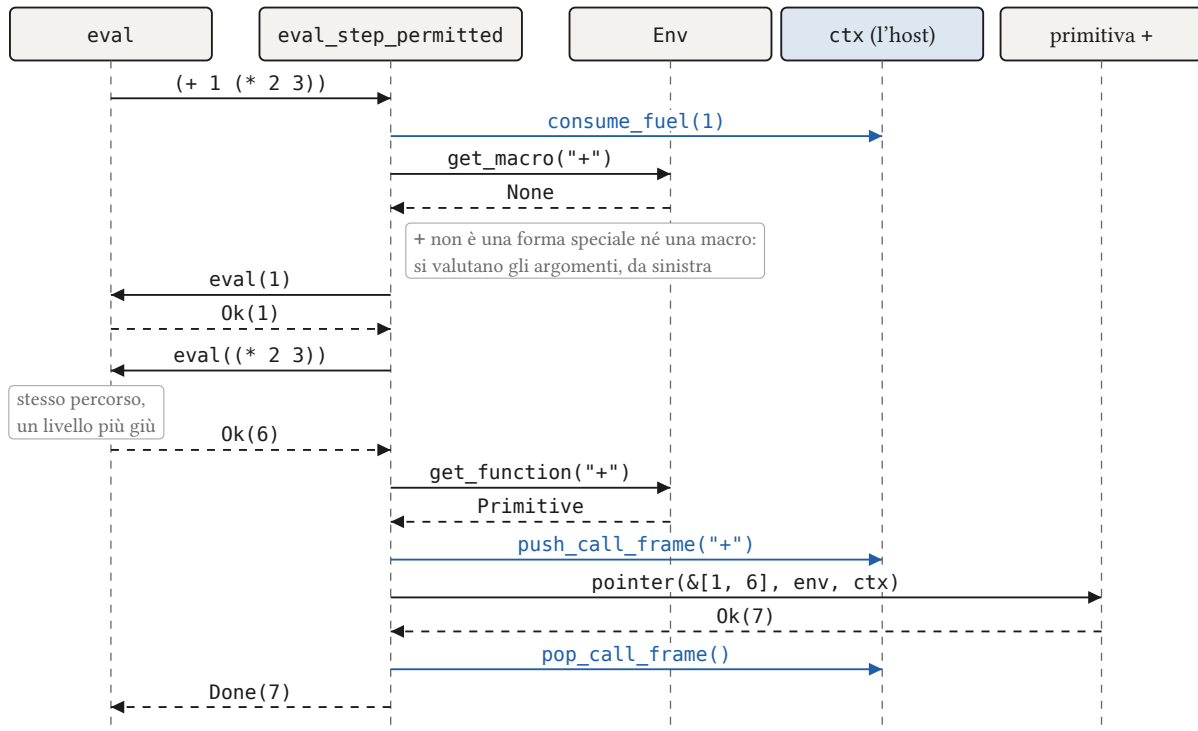
- Il *lettore* (`Parser`) produce un albero di `LispExp`. Le liste del codice sono `LispExp::Form`, cioè vettori, non catene di cons (Sezione 3.9).
- Il *valutatore* (`eval`) riduce l'albero in un ciclo, il trampolino, che esegue le chiamate in coda senza consumare lo stack di Rust (Sezione 6.1).
- L'*ambiente* (`Env`) è una catena di tabelle: ogni chiamata di funzione e ogni `let` ne aggiunge un anello (Sezione 5).
- Il *contesto* è l'host. Il valutatore lo chiama a ogni passo per far pagare il carburante, e in pochi altri punti per la pila delle chiamate, le pulizie e i nuovi thread (Sezione 11). Le primitive dell'host non passano dal contesto: l'host le registra nella stessa radice delle primitive di base, e il valutatore le trova per nome come tutte le altre.
- Il risultato è un `Result`. Il ramo `Err` porta sia gli errori sia le uscite non locali — `throw`, `signal` e `yield` — perché in Rust srotolare lo stack si fa propagando un `Err` con `?` (Sezione 7).

Il punto d'ingresso per un host è quindi sempre lo stesso:

```
let env = bootstrap_vm(&ctx)?;           // ambiente radice + primitive
let ast = Parser::new("(+ 1 2)").next()?; // testo -> LispExp
let value = eval(&ast, env.clone(), &ctx)?; // LispExp -> valore
```

2.4 Primo esempio: una chiamata a una primitiva

Il diagramma di sequenza segue la valutazione di `(+ 1 (* 2 3))`. Il tempo scorre verso il basso; le frecce continue sono chiamate, quelle tratteggiate risposte.



Il ciclo di `eval` riceve `Done(7)` e restituisce `Ok(7)`. Ogni passo di `eval_step_permitted` costa un'unità di carburante, e qui i passi sono cinque: la forma esterna, `1`, la forma interna, `2` e `3`. Le teste `+` e `*` non si valutano: sono nomi cercati nello spazio delle funzioni. Un simbolo in testa a una forma non è mai una variabile.

2.5 Secondo esempio: una funzione ricorsiva in coda

La funzione seguente calcola il fattoriale con un accumulatore. La chiamata ricorsiva è l'ultima cosa che il ramo «altrimenti» dell' `if` fa, cioè è in *posizione di coda*.

```
(defun fatt (n acc)
  (if (= n 0) acc (fatt (- n 1) (* n acc))))
(fatt 3 1) ; => 6
```

La tabella segue il ciclo di `eval` (il trampolino, Sezione 6.1) durante la valutazione di `(fatt 3 1)`. Ogni riga è un giro del ciclo: la forma corrente, l'ambiente in cui va valutata e ciò che il passo restituisce.

N.	Forma corrente	Amb.	Esito del passo
1	<code>(fatt 3 1)</code>	radice	argomenti valutati (3, 1); nuovo ambiente E1 figlio della radice con <code>n=3</code> , <code>acc=1</code> ; il corpo ha una sola forma, quindi <code>TailCall((if ...), E1)</code>

N.	Forma corrente	Amb.	Esito del passo
2	(if (= n 0) acc (fatt ...))	E1	la condizione è falsa: TailCall((fatt (- n 1) (* n acc)), E1)
3	(fatt (- n 1) (* n acc))	E1	argomenti 2 e 3; E2 figlio della radice con n=2, acc=3; TailCall((if ...), E2)
4	(if ...)	E2	TailCall((fatt ...), E2)
5	(fatt ...)	E2	E3 con n=1, acc=6; TailCall((if ...), E3)
6, 7	(if ...), (fatt ...)	E3	E4 con n=0, acc=6; TailCall((if ...), E4)
8	(if (= n 0) acc ...)	E4	la condizione è vera: TailCall(acc, E4)
9	acc	E4	Done(6) : il ciclo restituisce 0k(6)

Il ciclo non cresce: ogni chiamata in coda *sostituisce* la coppia (forma, ambiente) corrente invece di aggiungere un frame allo stack di Rust. Gli ambienti E1...E4 non si annidano l'uno nell'altro: ognuno è figlio dell'ambiente in cui `fatt` è stata *definita*, la radice (Sezione 5.5). Gli argomenti, invece, sono valutati con chiamate ricorsive a `eval`, perché il loro valore serve prima della chiamata: è la differenza tra una ricorsione in coda, che non consuma stack, e una che non lo è (Sezione 6.1).

2.6 Gli invarianti su cui si regge tutto

Sei regole attraversano tutto il codice. Ogni scelta descritta nei capitoli successivi ne è una conseguenza.

1. **Il codice è Form, i dati sono Cons.** Il lettore produce vettori, perché il valutatore accede alle liste di codice per posizione; i valori sono catene di celle, perché `car` e `cdr` devono costare O(1) (Sezione 3.9).
2. **Clonare non copia.** Ogni contenitore sta dietro un `Arc`: il valutatore clona forme, valori e ambienti a ogni passo, e ogni clone costa un incremento di un contatore.
3. **Un passo, un'unità di carburante.** Il valutatore paga all'ingresso di ogni passo, prima di sapere che cosa farà (Sezione 9).
4. **Errori, salti e sospensioni sono tutti Err.** Lo srotolamento dello stack è affidato a `?`, e nessuna funzione intermedia deve sapere che cosa la sta attraversando (Sezione 7).
5. **Nessun lock resta preso mentre gira codice Lisp.** La ricerca negli ambienti prende un lock alla volta; `resume` toglie la pila della fiber e rilascia il lock prima di eseguirla.
6. **L'interprete non conosce l'host.** Tutto ciò che dipende dall'host passa dai metodi di `LispContext` o dalle primitive che l'host registra.

3 Il modello dei dati

3.1 LispExp, il tipo di tutto

Ogni valore e ogni pezzo di codice è un `LispExp<T>`. Il parametro `T` è il contesto: compare qui perché alcune varianti (lambda, primitive, fiber) racchiudono codice che, eseguito, avrà bisogno dell'host.

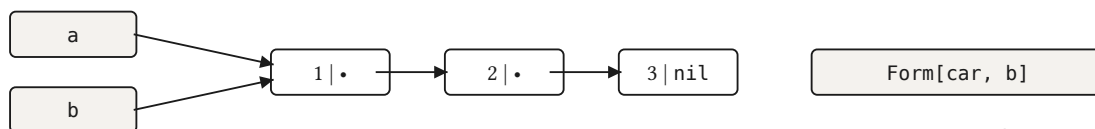
da `lispexp.rs` — estratto semplificato

```
pub enum LispExp<T: LispContext> {
    Form(Arc<Vec<LispExp<T>>>), // sintassi: ciò che produce il lettore
    Cons(Arc<ConsCell<T>>),      // dati: una cella di lista
    Vector(Arc<Vec<LispExp<T>>>),
    Map(Arc<HashMap<String, LispExp<T>>>),
    Number(f64),
    Symbol(Arc<String>),          // internato nell'obarray
    String(Arc<String>),
    Lambda(Arc<Lambda<T>>),
    Primitive { pointer: LispPrimitive<T>, doc: Arc<Cow<'static, str>> },
    Atom(SharedAtom<T>),          // Arc<RwLock<LispExp<T>>>
    Fiber(SharedFiber<T>),        // Arc<RwLock<FiberState<T>>>
}
```

Variante	Contenuto	Note
Form	<code>Arc<Vec<LispExp>></code>	Un nodo di <i>sintassi</i> : ciò che il lettore produce e <code>eval</code> smista. Mai un valore a tempo di esecuzione.
Cons	<code>Arc<ConsCell></code>	Una lista di <i>dati</i> . Termina in <code>nil</code> (lista propria) o in altro (lista puntata).
Vector	<code>Arc<Vec<LispExp>></code>	<code>[a b c]</code> . Valutare un vettore ne valuta gli elementi.
Map	<code>Arc<HashMap<String, LispExp>></code>	<code>{chiave valore ...}</code> . Chiavi simbolo o stringa, conservate come testo; valutarla ne valuta i valori. L'ordine non è conservato.
Number	<code>f64</code>	L'unico tipo numerico. Si stampa come intero quando lo è (<code>3</code> , non <code>3.0</code>).
Symbol	<code>Arc<String></code> , internato	Due simboli con lo stesso nome condividono lo stesso <code>Arc</code> .
String	<code>Arc<String></code>	Immutabile; clonarla costa un incremento del contatore.
Lambda	<code>Arc<Lambda></code>	Una chiusura: parametri, corpo e ambiente catturato. Sono <code>Lambda</code> anche le funzioni di <code>defun</code> e le macro.
Primitive	puntatore a <code>fn</code> + doc	Una funzione scritta in Rust.
Atom	<code>SharedAtom</code>	L'unico contenitore mutabile; attraversa i thread.
Fiber	<code>SharedFiber</code>	Un programma sospendibile (Sezione 8).

3.2 Clonare non copia

Tutti i contenitori stanno dietro un `Arc`: clonare un `LispExp` non copia mai i dati, incrementa un contatore. Per questo il valutatore può clonare forme e valori a ogni passo senza preoccuparsi del costo. La figura mostra due variabili che condividono la stessa lista e una forma che nomina una delle due.



la forma contiene il simbolo `b`, non la lista:
 è codice, e `b` sarà cercato quando la forma è valutata

(setq a '(1 2 3)) · (setq b a): un solo Arc<ConsCell> per cella, condiviso. Nessuna copia.
 (eq a b) è vero perché è la stessa cella; (equal a b) lo sarebbe anche per due copie distinte.

La condivisione è sicura perché le liste sono *immutabili*: non esistono `setcar` né `setcdr`, e nessuna primitiva modifica una cella esistente. Le primitive che «cambiano» una lista (`append`, `reverse`, `add-to-list`) ne costruiscono una nuova. L'unico contenitore che si può modificare sul posto è l'atomo, che per questo ha un lock.

3.3 Costruire e scorrere le liste

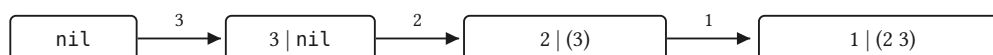
Tre costruttori creano liste di dati, e tutti piegano un vettore *da destra*, perché una lista si costruisce dalla coda verso la testa:

da `lispexp.rs` — estratto semplificato

```
pub fn proper_list(items: Vec<LispExp<T>>) -> LispExp<T> {
    items.into_iter().rev()
        .fold(LispExp::nil(), |cdr, car| LispExp::cons(car, cdr))
}
pub fn improper_list(items: Vec<LispExp<T>>, tail: LispExp<T>) -> LispExp<T> {
    items.into_iter().rev().fold(tail, |cdr, car| LispExp::cons(car, cdr))
}
```

vec![1, 2, 3] rev() → 3, 2, 1

ogni passo della piega: cons(elemento, accumulato)



Per scorrere una lista ci sono due modi. `iter()` restituisce un `ConsIter` che produce i `car` uno alla volta e si ferma al primo `cdr` che non è una cella: una lista puntata viene vista come il suo prefisso proprio. `split_list()` raccoglie invece (elementi, coda finale), e la coda dice se la lista era propria (`nil`) o puntata. Chi deve distinguere i due casi, come `data_to_form` o la stampa, usa `split_list`.

3.4 nil, t e la verità

`nil` e `t` sono simboli. Il loro Arc è conservato in una `OnceLock`, per cui `LispExp::nil()` e `LispExp::t()` non toccano mai un lock: il valutatore costruisce `nil` all'uscita di quasi ogni forma, e non deve pagare l'obarray.

da `lispexp.rs` — estratto semplificato

```
pub fn is_nil(&self) -> bool {
    match self {
        LispExp::Symbol(s) => s.as_str() == "nil",
        LispExp::Form(l) => l.is_empty(),      // la forma vuota ()
        _ => false,
    }
}
pub fn is_truthy(&self) -> bool { !self.is_nil() }
```

È falso soltanto `nil`, insieme alla forma vuota `()`; tutto il resto, compresi `0` e la stringa vuota, è vero. I simboli che iniziano con `:` (le *keyword*) valgono sé stessi, come `nil` e `t`, senza bisogno di essere legati: il valutatore li riconosce prima di cercare la variabile (Sezione 6.3).

3.5 L'obarray

L'obarray è la tabella dei nomi dei simboli: `intern(nome)` restituisce l'unico `Arc<String>` per quel nome, così due simboli con lo stesso nome sono lo stesso puntatore e `eq` sui simboli è un confronto di puntatori.

da `lispexp.rs` — estratto semplificato

```
pub fn intern(name: String) -> Arc<String> {
    // il caso comune: un lock di lettura e il nome c'è già
    if let Some(found) = obarray().read().unwrap().get(&name) {
        return found.clone();
    }
    // solo la prima volta che un nome compare: il lock di scrittura
    obarray().write().unwrap()
        .entry(name.clone())
        .or_insert_with(|| Arc::new(name))
        .clone()
}
```

La tabella è una `static` del processo, condivisa da tutti gli interpreti e da tutti i thread. Può esserlo perché contiene soltanto nomi e mai un `LispExp`: Rust non ammette `static` generiche, e una tabella che contenesse valori dovrebbe essere generica su `T`. Per lo stesso motivo le *proprietà* dei simboli non stanno nel simbolo ma nell'ambiente radice (Sezione 5.6). Nel ramo di scrittura, `entry(...).or_insert_with` gestisce anche il caso in cui un altro thread abbia inserito lo stesso nome tra il rilascio del lock di lettura e l'acquisizione di quello di scrittura: vince il primo, e tutti ricevono il suo `Arc`.

3.6 Uguaglianza

Il linguaggio ha due nozioni di uguaglianza, e il codice Rust ne ha una terza su cui la seconda si appoggia.

3.6.1 *PartialEq* per *LispExp*

È scritto a mano, variante per variante:

Coppia	Quando sono uguali
<code>Cons</code> , <code>Cons</code>	Confronto strutturale, <i>iterativo</i> lungo i <code>cdr</code> : un ciclo avanza sulle due catene, si ferma con <code>true</code> se le due celle correnti sono lo stesso <code>Arc</code> (<code>Arc::ptr_eq</code> , che evita anche di riscorrere una coda condivisa), con <code>false</code> al primo <code>car</code> diverso. I <code>car</code> sono confrontati con una chiamata ricorsiva.
<code>Lambda</code> , <code>Lambda</code>	Solo se sono la stessa chiusura (<code>Arc::ptr_eq</code>). Due chiusure scritte uguali sono diverse: confrontarle strutturalmente vorrebbe dire confrontare gli ambienti catturati.
<code>Form</code> , <code>Vector</code> , <code>Map</code>	Confronto strutturale degli elementi (ricorsivo).
<code>Number</code>	<code>f64 ==</code> : quindi <code>NaN</code> non è uguale nemmeno a sé stesso.
<code>Symbol</code> , <code>String</code>	Confronto del testo.
<code>Atom</code>	Stesso <code>Arc</code> (<code>SharedAtom</code> confronta con <code>Arc::ptr_eq</code>).
<code>Fiber</code>	<code>SharedFiber</code> confronta gli indirizzi dei due involucri con <code>std::ptr::eq</code> ($\rightarrow$ INT-8).
<code>Primitive</code>	Stesso puntatore a funzione.
varianti diverse	Mai uguali: <code>(1 2)</code> come <code>Form</code> e come <code>Cons</code> sono diversi.

3.6.2 eq, eql ed equal

Primitiva	Semantica
equal	È PartialEq: il confronto strutturale profondo descritto sopra.
eq, eql	Identità. Numeri e simboli si confrontano per valore; nil con nil (in entrambe le forme, simbolo e ()); forme, cons, vettori, mappe e stringhe sono eq solo se sono la stessa allocazione (Arc::ptr_eq). Per le altre varianti (lambda, atomi, fiber, primitive) non c'è un ramo, e il risultato è nil anche per un oggetto confrontato con sé stesso (→ INT-7).
=, <, ...	Solo numeri; accettano un numero qualunque di argomenti e controllano l'intera catena (compare_chain).

Espressione	eq	equal
1 e 1.0	t	t
'a e 'a	t	t
"ab" e "ab" lette separatamente	nil	t
(list 1 2) e (list 1 2)	nil	t
x e x, con x una lista	t	t
il valore di (spawn ...) e nil	t	nil : una forma vuota e il simbolo nil sono varianti diverse (→ INT-18)

3.7 Stampa

Debug è scritto a mano e produce sintassi Lisp: è ciò che finisce nell'area dei messaggi, nei log e nei risultati di M-: .

Variante	Come si stampa
Form, Cons	(a b c) ; una lista puntata (a b . c) . Le due rappresentazioni si stampano allo stesso modo: la distinzione è interna.
Vector, Map	[a b], {chiave valore} .
Number	Come intero se è finito, intero e in valore assoluto sotto 10^{15} (3 , -42); altrimenti con il formato di f64 (1.5 , 1e300 , inf).
String	Tra virgolette, con gli escape di Rust: "a\nb" .
Lambda	#<lambda (x &optional y &rest z)> .
Primitive, Atom	#<primitive> , #<atom> . L'atomo non mostra il contenuto: è l'unico valore che può contenere sé stesso, e stamparlo potrebbe non finire.
Fiber	#<fiber> , #<fiber done> , oppure #<fiber running> se il lock è occupato (la stampa usa try_read e non aspetta).

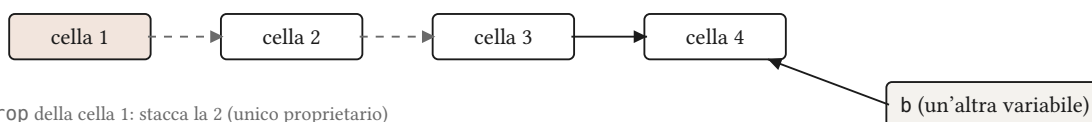
lisp_display (in base/mod.rs) è invece la conversione «per le persone», usata da format con %s, da message e da insert: una stringa è il suo testo senza virgolette, un simbolo il suo nome, un numero passa da format_number, tutto il resto come lo stampa Debug. format_number stampa come intero ogni numero finito e intero, senza la soglia di 10^{15} di Debug (→ INT-14).

3.8 Liberare una lista lunga

Una lista di un milione di elementi è una catena di un milione di `Arc`. Lasciata alla distruzione automatica, ogni cella libererebbe la propria coda dentro il proprio `drop`, e la profondità della ricorsione sarebbe la lunghezza della lista: lo stack si esaurirebbe e il processo terminerebbe. `ConsCell` ha quindi un `Drop` che scollega la catena in un ciclo:

da `types.rs` — estratto semplificato

```
impl<T: LispContext> Drop for ConsCell<T> {
    fn drop(&mut self) {
        let mut next = std::mem::replace(&mut self.cdr, LispExp::nil());
        while let LispExp::Cons(cell) = next {
            match Arc::try_unwrap(cell) {
                // l'ultimo proprietario: stacca la coda prima che la cella muoia
                Ok(mut owned) => {
                    next = std::mem::replace(&mut owned.cdr, LispExp::nil())
                }
                // qualcun altro tiene viva la coda: ci penserà lui
                Err(_) => break,
            }
        }
    }
}
```

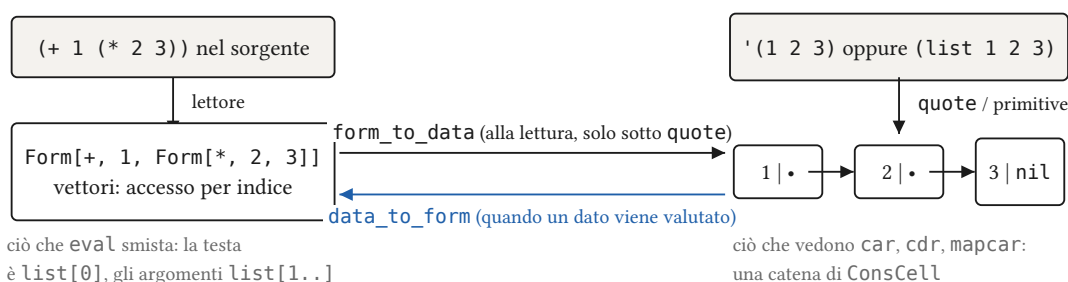


drop della cella 1: stacca la 2 (unico proprietario)
e la libera, stacca la 3 e la libera; la 4 è tenuta viva
da b: `try_unwrap` fallisce e il ciclo si ferma

Il ciclo scende solo lungo i `cdr`. Una lista annidata nei `car` (ogni elemento è la lista successiva) viene ancora liberata per ricorsione (→ INT-4).

3.9 Sintassi e dati: `Form` contro `Cons`

La distinzione più importante del modello è tra le liste che sono *codice* e quelle che sono *dati*.



Il lettore produce `Form` perché il valutatore accede alle liste di codice per posizione: un vettore rende immediati `list[0]` e `list[1..]`. Le liste *valori* sono catene di `Cons`, come in ogni Lisp: `car` e `cdr` costano $O(1)$ e le code si possono condividere. Due funzioni convertono da una rappresentazione all'altra.

3.9.1 form_to_data

da `parser.rs` — estratto semplificato

```
pub fn form_to_data<T: LispContext>(exp: &LispExp<T>) -> LispExp<T> {
    match exp {
        LispExp::Form(items) =>
            LispExp::proper_list(items.iter().map(form_to_data).collect()),
        LispExp::Vector(items) =>
            LispExp::vec(items.iter().map(form_to_data).collect()),
        LispExp::Map(m) =>
            LispExp::map(m.iter().map(|(k, v)| (k.clone(), form_to_data(v))).collect()),
        other => other.clone(),
    }
}
```

La chiama il lettore, una volta sola, su ciò che sta sotto un `'`: `'(1 (2 3))` è già una catena di cons quando `quote` la restituisce, e la conversione non si ripete a ogni valutazione. Una lista puntata scritta nel sorgente è già una `Cons` (il lettore la costruisce con `improper_list`) e passa invariata.

3.9.2 data_to_form

da `utils.rs` — estratto semplificato

```
pub(super) fn data_to_form<T>(exp: &LispExp<T>) -> Result<LispExp<T>, EvalError<T>> {
    match exp {
        LispExp::Cons(_) => {
            let (items, tail) = exp.split_list();
            if !tail.is_nil() { return Err(EvalError::InvalidFunctionCall); }
            // (quote X) costruita a mano: X deve restare un dato
            if items.len() == 2
                && matches!(&items[0], LispExp::Symbol(s) if s.as_str() == "quote")
            {
                return Ok(LispExp::form(vec![items[0].clone(), items[1].clone()]));
            }
            let mut form = Vec::with_capacity(items.len());
            for item in &items { form.push(data_to_form(item)?); }
            Ok(LispExp::form(form))
        }
        LispExp::Vector(items) => /* stessa cosa, elemento per elemento */,
        other => Ok(other.clone()),
    }
}
```

Fa il percorso inverso quando un *dato* viene valutato: `(eval (list '+ 1 2))` e ogni espansione di macro, che produce dati, passano da qui. Tre dettagli:

- una lista puntata non può diventare codice e dà `InvalidFunctionCall`;
- `(quote X)` costruita a mano viene ricostruita senza convertire `X`, perché `quote` deve restituire `X` come dato; convertirlo lo trasformerebbe in sintassi, e `quote` restituirebbe un nodo `Form` come valore;
- le mappe non vengono convertite: i loro valori restano dati.

Una primitiva che restituisce una lista a Lisp deve usare `LispExp::proper_list` (dati), mai `LispExp::form` (sintassi). `LispExp::form` è riservato al lettore, alle espansioni di macro e a `data_to_form`. Vale anche al contrario: `(funcall '(lambda (x) x) 3)` non funziona, perché una lista citata è un dato e non una chiusura; una chiusura nasce solo valutando la forma `lambda`.

3.10 Le strutture di `types.rs`

Tipo	Ruolo
<code>ConsCell { car, cdr }</code>	Una cella di lista; <code>Drop</code> iterativo lungo i <code>cdr</code> .
<code>ConsIter</code>	L'iteratore restituito da <code>iter()</code> : tiene un cursore (<code>LispExp</code>) e produce i <code>car</code> finché il cursore è una <code>Cons</code> .
<code>Lambda</code>	<code>params</code> (obbligatori), <code>optionals</code> (<code>&optional</code>), <code>rest</code> (<code>&rest</code>), <code>body: Vec<LispExp></code> , <code>env: Arc<Env></code> catturato, <code>doc: Option<Arc<String>></code> . È la funzione di <code>defun</code> , la chiusura di <code>lambda</code> e la macro di <code>defmacro</code> : cambiano solo lo spazio dei nomi in cui viene legata e il modo in cui è chiamata.
<code>SharedAtom</code>	<code>Arc<RwLock<LispExp>></code> . Si confronta per identità.
<code>Frame</code>	Un blocco sospeso di una fiber: <code>Body { forms, from, env }</code> (il resto di un corpo) o <code>While { condition, forms, from, env }</code> (il resto di un'iterazione, poi il ciclo).
<code>FiberState</code>	<code>pending: Vec<Frame></code> (il più interno per primo) e <code>is_done</code> .
<code>SharedFiber</code>	<code>Arc<RwLock<FiberState>></code> : più thread possono avere la stessa fiber, uno solo la esegue.
<code>LispPrimitive<T></code>	<code>fn(&[LispExp<T>], Arc<Env<T>>, &T) -> Result<LispExp<T>, EvalError<T>></code> Riceve gli argomenti già valutati, l'ambiente del <i>chiamante</i> e il contesto.

Lambda, costruita da

```
(defun f (a &optional b &rest c) "doc" ...)
  params: ["a"] · optionals: ["b"] · rest: Some("c")
  body: le forme dopo la docstring
  doc: Some("doc")
  env: l'ambiente in cui defun è stata valutata
```

ogni chiamata crea `Env::new_child(&lambda.env)`:
lo scope è quello della definizione, non del chiamante

cattura

Env di definizione
(di solito la radice)

4 Il lettore

Il lettore trasforma il testo in un albero di `LispExp`. È fatto di due parti che collaborano: un *lexer* a stati, che legge un carattere alla volta e produce *token*, e un *parser* a discesa ricorsiva, che legge i token e costruisce le espressioni. Entrambe stanno in `parser.rs`, nella struttura `Parser`.

4.1 L'interfaccia

```
let mut parser = Parser::new(testo); // non legge ancora nulla
let prima = parser.next()?;          // la prima espressione completa
let seconda = parser.next()?;         // la successiva, e così via
// alla fine del testo: Err(ParserError::VoidExp)
```

`next()` restituisce una espressione completa per chiamata. Chi deve leggere un file intero può chiamarla in un ciclo finché riceve `VoidExp`, oppure avvolgere il testo in `(progn ...)` e leggere una sola espressione, che è ciò che fa `eval_file` nell'editor. `eval-string` legge e valuta soltanto la prima espressione della stringa.

4.2 Lo stato del Parser

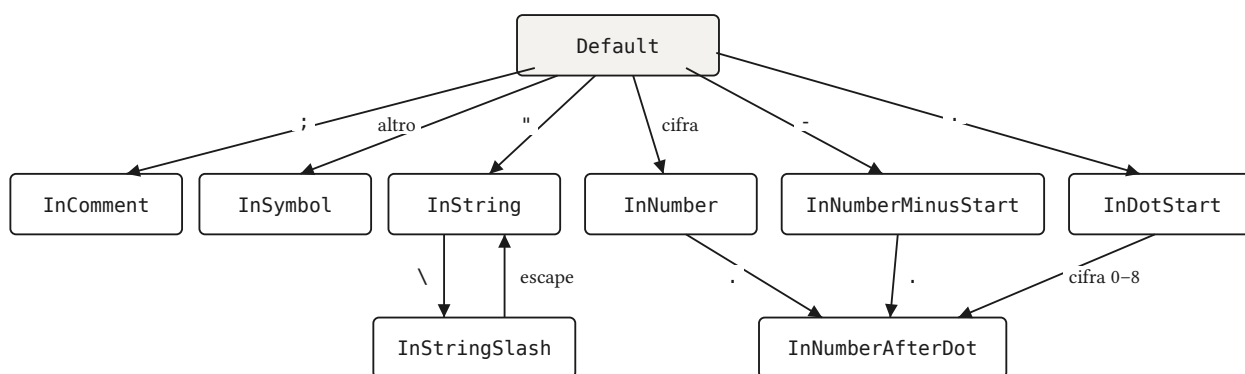
da `parser.rs` — estratto semplificato

```
pub struct Parser<'source> {
    source: Peekable<Chars<'source>>, // i caratteri, uno di anticipo
    token: String,                     // il token in costruzione
    current_token: Token,               // il token di anticipo del parser
    parens_stack: Vec<Token>,           // le aperture non ancora chiuse
    lexer_state: ParserLexerState,      // dove si trova il lexer in un token
}
```

Token	Testo che lo produce
LParen, RParen	(e)
LSquared, RSquared	[e]
LBracket, RBracket	{ e }
Quote, BackQuote	' e `
Comma, CommaAt	, e ,@
Dot	un . isolato
Number(f64), String, Symbol	gli atomi
Void	la fine dei token (prodotto da <code>advance_token</code> , non dal lexer)
Uninitialized	lo stato iniziale di <code>current_token</code> , prima della prima lettura

4.3 Il lexer

`next_token` guarda il carattere successivo con `peek()`, decide che cosa fare in base allo stato corrente e, nella maggior parte dei casi, lo consuma con `next()` in fondo al ciclo. Quando un carattere chiude il token in corso senza farne parte — una parentesi dopo un simbolo, per esempio — lo stato torna a `Default` senza consumarlo: la chiamata successiva lo troverà di nuovo e lo tratterà come il primo carattere di un nuovo token.



Ogni stato torna a Default quando emette il suo token. Da InNumberMinusStart una cifra porta a InNumber; dagli stati dei numeri una lettera porta a InSymbol (1+, ->), tranne da InNumberAfterDot, che dà NumberParseError (1.5x). La tabella sotto è il riferimento completo.

Stato	Carattere letto → azione
Default	spazio, tab, \n, \r → si salta · ; → InComment · ' → Quote · ` → BackQuote · , → Comma, oppure CommaAt se seguita da @ · ([{ → empila l'apertura su parens_stack e la emette ·)] } → disimpila: se l'apertura corrisponde emette la chiusura, altrimenti UnbalancedRParen (o ...RSquared, ...RBracket) · " → InString · - → InNumberMinusStart · cifra → InNumber · . → InDotStart · altro → InSymbol
InComment	\n → Default · ogni altro carattere si salta
InSymbol	spazio, tab, \n → emette Symbol e consuma il separatore · parentesi di ogni tipo → emette Symbol senza consumare · ; → emette Symbol, poi InComment · \r → ignorato · altro → si accumula (anche ", ', ,)
InString	" → emette String · \ → InStringSlash · altro → si accumula, a capo compreso
InStringSlash	\ " e \\ → il carattere · \n, \t, \r → a capo, tab, ritorno · \e → ESC · \0 → NUL · ogni altro \x → si accumulano entrambi i caratteri; poi InString
InNumber	cifra → si accumula · . → InNumberAfterDot · separatore, parentesi o ; → converte con str::parse::<f64> ed emette Number · altro → InSymbol
InNumberMinusStart	cifra → InNumber · . → InNumberAfterDot · separatore, parentesi o ; → emette Symbol("-") · altro → InSymbol
InNumberAfterDot	cifra → si accumula · separatore, parentesi o ; → converte ed emette Number · altro → NumberParseError
InDotStart	cifra da 0 a 8 → InNumberAfterDot · separatore, parentesi o ; → emette Dot · altro, compreso 9 → InSymbol (→ LET-4)
fine dell'input	Default, InComment → nessun token · InSymbol, InNumberMinusStart → emette Symbol (→ LET-1) · InNumber, InNumberAfterDot → emette Number · InDotStart → emette Dot · InString, InStringSlash → UnclosedString

Da queste regole discendono le convenzioni che contano quando si scrive Lisp per rsedit:

Elemento	Regola
Delimitatori	Spazio, tabulazione, <code>\n</code> , parentesi di ogni tipo; <code>;</code> apre un commento fino a fine riga anche attaccato a un simbolo (<code>foo;commento</code>).
Numeri	Cifre con un punto facoltativo, eventualmente precedute da <code>-</code> : <code>42</code> , <code>-3</code> , <code>1.5</code> , <code>.5</code> . Una cifra seguita da lettere diventa un simbolo (<code>1+</code>); <code>1.5x</code> è un errore. Non esiste la notazione esponenziale: <code>1e20</code> è un simbolo.
<code>-</code>	Da solo, o seguito da lettere, è un simbolo (<code>-</code> , <code>-></code> , <code>-foo</code>); seguito da cifre è un numero.
Stringhe	Escape riconosciuti: <code>\</code> , <code>\\</code> , <code>\n</code> , <code>\t</code> , <code>\r</code> , <code>\e</code> (ESC), <code>\0</code> . Ogni altro <code>\x</code> resta com'è, backslash compreso: così le espressioni regolari possono usare <code>\b</code> , <code>\s</code> , <code>\.</code> con un solo backslash.
Simboli	Qualunque sequenza di caratteri che non siano delimitatori: <code>*scratch*</code> , <code>C-x</code> , <code>string=</code> , <code>&rest</code> , <code>:keyword</code> .

4.4 Il parser

Il parser tiene un solo token di anticipo, `current_token`, e lo sostituisce con il successivo chiamando `advance_token`, che trasforma la fine dei token in `Token::Void`. La grammatica, in forma compatta:

```

espr    ::= atomo | lista | vettore | mappa
          | "'" espr | "\"" espr | "," espr | ",@" espr
lista   ::= "(" espr* ")" | "(" espr+ "." espr ")"
vettore ::= "[" espr* "]"
mappa   ::= "{" (chiave espr)* "}"      chiave ::= simbolo | stringa
atomo   ::= numero | stringa | simbolo

```

Ogni regola è una funzione. `next` guarda `current_token` e sceglie:

da `parser.rs` — estratto semplificato

```

pub fn next<T: LispContext>(&mut self) -> Result<LispExp<T>, ParserError> {
    match self.current_token.clone() {
        Token::Symbol(s) => { self.advance_token()?; Ok(LispExp::symbol(s)) }
        Token::String(s)  => { self.advance_token()?; Ok(LispExp::string(s)) }
        Token::Number(n)  => { self.advance_token()?; Ok(LispExp::number(n)) }
        Token::LParen      => { self.advance_token()?; self.parse_list() }
        Token::LSquared    => { self.advance_token()?; self.parse_vector() }
        Token::LBracket    => { self.advance_token()?; self.parse_map() }
        Token::Uninitialized => { self.advance_token()?; self.next() }
        Token::Quote => {
            self.advance_token()?;
            let quoted = self.next()?;
            Ok(LispExp::form(vec![LispExp::symbol("quote"), form_to_data(&quoted)]))
        }
        Token::BackQuote => /* Form[backquote, next()] */,
        Token::Comma      => /* Form[unquote, next()] */,
        Token::CommaAt    => /* Form[unquote-splicing, next()] */,
        Token::Void => Err(ParserError::VoidExp),
        _ => unreachable!(...), // Dot, RParen, RSquared, RBracket
    }
}

```

L'ultimo ramo raccoglie i token che non possono cominciare un'espressione: un `.` fuori dalla coda di una lista, o una chiusura subito dopo un prefisso come in `'(a )` (→ LET-2).

4.4.1 Liste, vettori e mappe

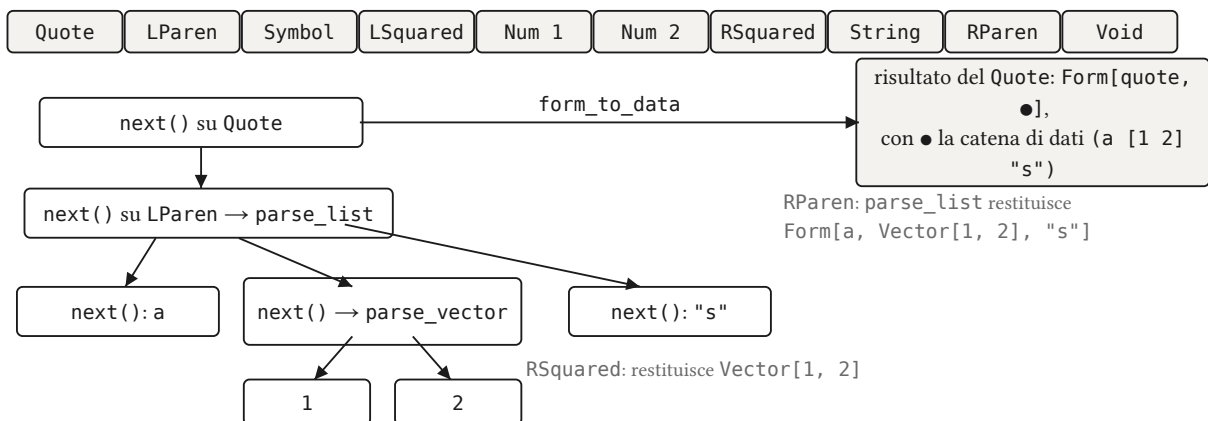
da `parser.rs` — estratto semplificato

```
fn parse_list<T: LispContext>(&mut self) -> Result<LispExp<T>, ParserError> {
    let mut list = vec![];
    while self.current_token != Token::Void {
        match self.current_token {
            Token::RParen => { self.advance_token()?; return Ok(LispExp::form(list)); }
            Token::Dot => {
                if list.is_empty() { return Err(ParserError::UnexpectedDot); }
                self.advance_token()?;
                if self.current_token == Token::RParen || self.current_token == Token::Void {
                    return Err(ParserError::UnexpectedDot);
                }
                let tail = self.next()?;
                if self.current_token != Token::RParen {
                    return Err(ParserError::MalformedDottedList);
                }
                self.advance_token()?;
                return Ok(LispExp::improper_list(list, tail));
            }
            _ => list.push(self.next()),
        }
    }
    Err(ParserError::UnclosedList)
}
```

Una lista normale diventa una `Form`; una lista puntata (`a b . c`) diventa direttamente una catena di `Cons` che termina in `c`, perché non è codice valutabile. `parse_vector` è lo stesso ciclo senza il punto. `parse_map` alterna due fasi: in quella della chiave accetta solo un simbolo o una stringa (`InvalidMapKey` altrimenti) e ne conserva il *testo* in quella del valore legge un'espressione qualsiasi, e una `}` al posto del valore è `MapKeyMissingValue`. Due chiavi uguali tengono l'ultimo valore.

4.4.2 Un esempio completo

La figura segue la lettura di `'(a [1 2] "s")`: in alto la sequenza dei token che il lexer produce, in basso l'albero delle chiamate del parser e ciò che ciascuna restituisce.



In alto i token, nell'ordine in cui il lexer li emette. Ogni chiamata a `next()` consuma il proprio token con `advance_token` e lascia in `current_token`

il primo token che non le appartiene: è così che `parse_list` vede la `RParen` dopo che `parse_vector` ha consumato la `RSquared`.

4.4.3 Gli errori del lettore

ParserError	Quando
<code>UnbalancedRParen</code> , <code>...RSquared</code> , <code>...RBracket</code>	Una chiusura che non corrisponde all'ultima apertura sulla <code>parens_stack</code> , o una chiusura senza aperture: <code>)</code> , <code>]</code> , <code>}]</code> .

ParserError	Quando
UnclosedList, UnclosedVector, UnclosedMap	Il testo finisce prima della chiusura (in uno stato in cui il lexer lo può dire, vedi (→ LET-1)).
UnclosedString	Il testo finisce dentro una stringa.
NumberParseError(testo)	Il testo di un numero non si converte in f64, o un numero con il punto è seguito da una lettera: 1.5x.
UnexpectedDot	Un punto all’inizio di una lista (. a) o subito prima della chiusura (a .).
MalformedDottedList	Più di un elemento dopo il punto: (a . b c).
InvalidMapKey, MapKeyMissingValue	Una chiave che non è un simbolo o una stringa; una chiave senza valore.
VoidExp	Il testo è finito: non è un errore vero, è il modo in cui chi legge in un ciclo capisce di aver letto tutto.

I prefissi ' e ` sono trasformati dal lettore. Una virgola dentro una forma *citata* (',x) è già un dato quando l'espansore del backquote la vede, e non viene espansa. Il commento in `defcommand (core/lisp/commands.lisp)` mostra come aggirare il problema.

5 Gli ambienti

Un ambiente associa nomi a valori. È l'unico posto in cui un simbolo acquista un significato: valutare `x` vuol dire cercare `x` in un ambiente, chiamare `(f 1)` vuol dire cercare `f` in un ambiente.

5.1 La struttura di `Env`

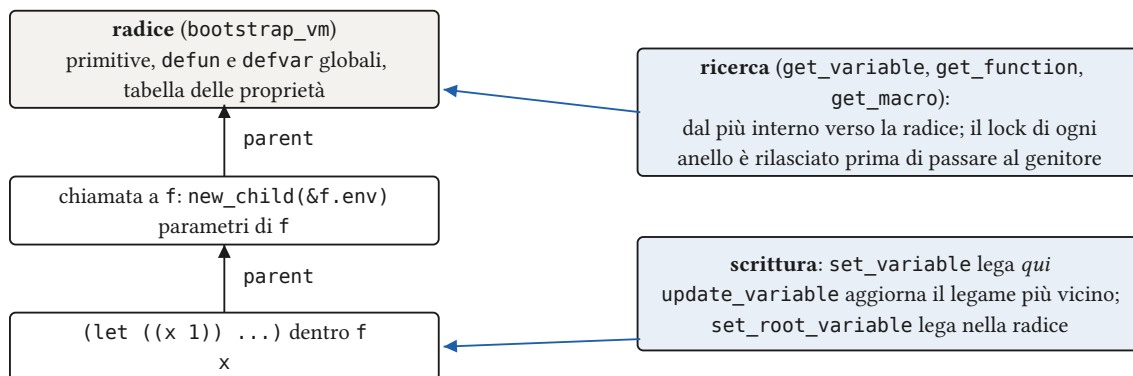
da `environment.rs` – estratto semplificato

```
pub struct Env<T: LispContext> {
    pub variables: RwLock<HashMap<String, LispExp<T>>>,
    pub functions: RwLock<HashMap<String, LispExp<T>>>,
    pub macros:    RwLock<HashMap<String, LispExp<T>>>,
    pub properties: Option<Box<PropertyTable<T>>>, // solo nella radice
    pub parent:     Option<Arc<Env<T>>>,
}
```

Un ambiente ha *tre spazi dei nomi* separati, come Emacs Lisp: un simbolo può essere insieme una variabile, una funzione e una macro senza che i tre legami interferiscano. `(setq sq 5)` e `(defun sq () 6)` convivono: `sq` vale 5 e `(sq)` vale 6. Una macro non è una funzione: `funcall` e `functionp` non la vedono. Ogni tabella ha il proprio `RwLock`, così due thread possono leggere lo stesso ambiente insieme e una scrittura blocca una sola tabella.

Gli ambienti formano una catena verso la radice. Ci sono due costruttori:

- `Env::new_root()` crea la radice, l'unico ambiente con la tabella delle proprietà; la crea `bootstrap_vm`, una volta per interpretare;
- `Env::new_child(&genitore)` crea un anello vuoto, senza tabella delle proprietà. Viene chiamato a ogni chiamata di funzione, a ogni `let` e `let*`, in ogni `dolist` e `dotimes`, nel gestore di `condition-case`, in una fiber e in uno `spawn`: per questo è tenuto il più leggero possibile.



5.2 La ricerca

Le tre ricerche sono la stessa funzione, parametrizzata sullo spazio dei nomi:

da `environment.rs` – estratto semplificato

```
fn lookup(&self, namespace: Namespace, name: &str) -> Option<LispExp<T>> {
    let mut env = self;
    loop {
        let found = env.table(namespace).read().unwrap().get(name).cloned();
        if found.is_some() { return found; } // il lock è già rilasciato qui
        env = env.parent.as_deref()?;       // radice superata: None
    }
}
```

Il ciclo è iterativo e prende *un lock alla volta*: il lock di lettura di un anello viene rilasciato, alla fine dell'istruzione, prima che si passi al genitore. Tenere un lock su tutta la catena insieme imporrebbe un ordine di acquisizione che il resto dell'editor non rispetta, e un thread che scrive in un anello esterno potrebbe bloccarsi contro uno che legge da un anello interno. Il valore trovato è clonato, cioè restituito come un nuovo Arc sullo stesso dato: chi lo riceve non tiene nulla dell'ambiente.

5.3 La scrittura

Ci sono tre modi di scrivere una variabile, e le forme speciali scelgono quale usare (Sezione 5.4):

Metodo	Che cosa fa
<code>set_variable(n, v)</code>	Lega <code>n</code> nell'ambiente <i>su cui è chiamato</i> , anche se un anello esterno ha già un legame con lo stesso nome: il nuovo legame lo nasconde.
<code>update_variable(n, v)</code>	Cerca il legame più vicino e lo sostituisce, restituendo <code>true</code> ; se nessun anello lo ha, non fa nulla e restituisce <code>false</code> . È ricorsivo lungo la catena: controlla con un lock di lettura se l'anello contiene il nome, poi prende il lock di scrittura per sostituirlo.
<code>set_root_variable(n, v)</code>	Risale fino alla radice e lega lì.

`set_function` e `set_macro` legano sempre nell'anello su cui sono chiamati.

5.4 Come lega ciascuna forma

Forma	Dove scrive
<code>setq</code>	<code>update_variable</code> : aggiorna il legame esistente più vicino. Se non ne trova, crea il legame <i>nell'ambiente corrente</i> (<code>set_variable</code>). Restituisce l'ultimo valore assegnato.
<code>let</code>	Valuta tutti i valori nell'ambiente esterno, poi li lega in un figlio nuovo.
<code>let*</code>	Lega uno alla volta nel figlio, così ogni valore vede i precedenti.
<code>defvar</code>	Lega nell'ambiente <i>corrente</i> solo se la variabile non è visibile da lì. La docstring, se c'è, va sempre nelle proprietà del simbolo.
<code>defconst</code>	Come <code>defvar</code> , ma assegna sempre.
<code>defun</code> , <code>defmacro</code>	Legano nell'ambiente corrente, nello spazio delle funzioni o delle macro.
<code>dolist</code> , <code>dotimes</code>	Creano un figlio con la variabile del ciclo e la aggiornano a ogni giro.
parametri di una chiamata	<code>bind_lambda_args</code> lega con <code>set_variable</code> nell'ambiente nuovo della chiamata.
primitive come <code>add-to-list</code>	Scrivono con <code>set_variable</code> nell'ambiente che ricevono, cioè quello del chiamante.

Poiché tutte queste forme scrivono nell'ambiente *in cui sono valutate*, ciò che accade dentro il corpo di una funzione resta locale alla chiamata:

- `(setq nuova 1)` su una variabile mai dichiarata crea un legame nell'ambiente della chiamata, che sparisce al ritorno (→ INT-3)
- `defun`, `defvar` e `defmacro` eseguiti dentro una funzione definiscono nomi visibili solo durante quella chiamata;
- `(add-to-list 'lista x)` chiamato in una funzione lega una nuova lista nell'ambiente della chiamata (→ INT-15).

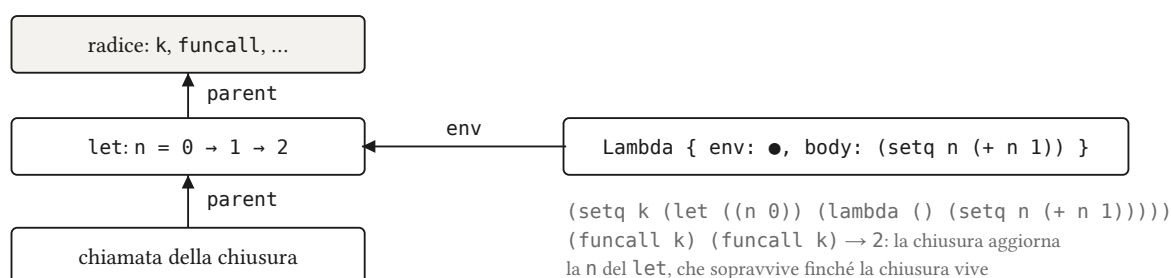
Per modificare lo stato globale da una funzione, la variabile deve esistere già nella radice: `setq` la trova risalendo la catena e la aggiorna lì. È così che funzionano i moduli in `core/lisp/`: dichiarano le variabili a livello di file, dove l'ambiente corrente è la radice.

5.5 Lo scope è lessicale

Una `Lambda` ricorda l'ambiente in cui è stata creata, e la sua chiamata crea un figlio di *quell'*ambiente, non di quello del chiamante. Le variabili globali sono visibili ovunque perché stanno nella radice, in cima a ogni catena. Il test `test_lexical_vs_dynamic_scoping` fissa la scelta.

```
(setq x 10)
(defun get-x () x)
(let ((x 99)) (get-x)) ; => 10 in rsedit, 99 in Emacs con scope dinamico
```

Il valore `10` si spiega seguendo le catene: il corpo di `get-x` viene valutato in un figlio dell'ambiente di *definizione*, la radice, e da lì la ricerca di `x` non passa mai dall'anello del `let`.



La figura mostra il caso opposto, una chiusura che *vuole* vedere un ambiente diverso dalla radice. La chiusura tiene in vita l'anello del `let` attraverso il proprio campo `env`, e ogni chiamata ne crea un figlio: `(setq n ...)` risale la catena con `update_variable` e trova la `n` del `let`. L'anello vive finché vive la chiusura: è il conteggio dei riferimenti degli `Arc` a decidere quando liberarlo. Se l'anello contiene a sua volta la chiusura (per esempio perché la si è assegnata a una variabile dello stesso `let`), i due si tengono in vita a vicenda ($\rightarrow$ INT-6).

5.6 Le proprietà dei simboli

`put` e `get` leggono e scrivono una tabella `simbolo -> chiave -> valore` che esiste soltanto nella radice. Da qualunque anello, `property_owner` risale la catena fino al primo ambiente che ha la tabella:

da `environment.rs` — estratto semplificato

```
fn property_owner(&self) -> &Env<T> {
    let mut env = self;
    loop {
        if env.properties.is_some() { return env; }
        match env.parent.as_deref() { Some(p) => env = p, None => return env }
    }
}
```

Le proprietà sono quindi globali e non hanno scope: una proprietà impostata dentro un `let` resta dopo. L'editor le usa per i dati che appartengono a un nome, per esempio la larghezza del tab di un modo, `(put 'rust-mode 'tab-width 4)`. La docstring di una variabile è la proprietà `variable-documentation` (la costante `VARIABLE_DOCUMENTATION`), letta da `variable-doc`.

5.7 Letture tipizzate ed elenchi di nomi

Due metodi esistono perché l'host legge spesso impostazioni da Lisp:

- `number_at_least(nome, minimo)` restituisce il numero solo se è finito e almeno `minimo`, così una stringa, un negativo o un infinito non arrivano mai a `Duration::from_secs_f64`;
- `flag(nome, default)` distingue *non legata* da `nil`: un modulo che non si è caricato non deve poter spegnere una funzionalità per omissione.

`variable_names`, `function_names` e `macro_names` raccolgono le chiavi di ogni anello, dal corrente alla radice, poi le ordinano e tolgono i duplicati: è ciò che restituiscono `all-variables`, `all-functions` e `all-macros`, e ciò su cui si basa il completamento di `M-x` nell'editor.

6 Il valutatore

Il valutatore sta in `eval.rs` ed è fatto di cinque funzioni che si chiamano in quest'ordine: `eval` (il ciclo), `eval_step` (il permesso di sospendersi), `eval_step_permitted` (il carburante e lo smistamento per tipo), `eval_special_form_or_call_step` (le forme speciali) ed `eval_macro_or_function_call_step` (le chiamate). Ognuna restituisce un `EvalStep`: o un valore finito, o la forma successiva da valutare.

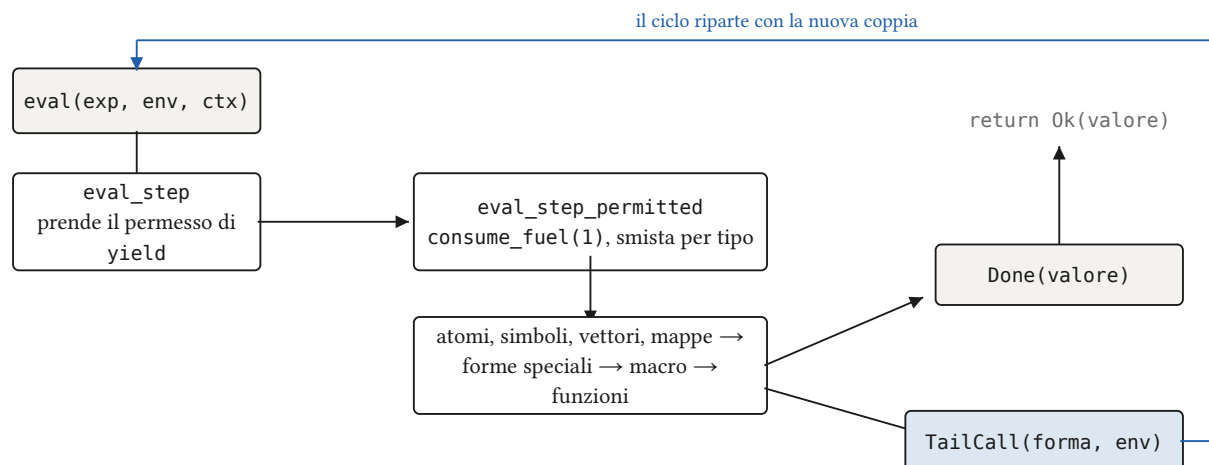
```
enum EvalStep<T: LispContext> {
    Done(LispExp<T>), // il passo ha prodotto il valore
    TailCall(LispExp<T>, Arc<Env<T>>), // resta da valutare questa forma, qui
}
```

6.1 Il trampolino

da `eval.rs` — estratto semplificato

```
pub fn eval<T: LispContext>(exp: &LispExp<T>, env: Arc<Env<T>>, ctx: &T)
-> Result<LispExp<T>, EvalError<T>>
{
    let mut current_exp = exp.clone();
    let mut current_env = env;
    loop {
        match eval_step(&current_exp, current_env.clone(), ctx)? {
            EvalStep::Done(result) => return Ok(result),
            EvalStep::TailCall(next_exp, next_env) => {
                current_exp = next_exp;
                current_env = next_env;
            }
        }
    }
}
```

Un passo che sa che il proprio risultato è il valore di un'altra forma non la valuta: la restituisce come `TailCall`, e il ciclo la valuta al giro successivo, *nello stesso frame Rust*. Così una chiamata in coda non consuma stack (lo si è visto con `fatt` nel Sezione 2).



Ogni altra valutazione annidata — gli argomenti di una chiamata, le condizioni, le forme non finali di un corpo — chiama `eval` ricorsivamente, e quindi apre un nuovo frame Rust. La tabella elenca tutte le posizioni di coda.

In posizione di coda (TailCall)	Valutato con una chiamata ricorsiva
l'ultima forma di <code>progn</code> , <code>let</code> , <code>let*</code> , <code>when</code> , <code>unless</code> , di una clausola di <code>cond</code> , del corpo di una funzione o di una chiusura chiamata in testa a una forma	le forme precedenti di quei corpi
i due rami di <code>if</code>	la condizione di <code>if</code> , i test di <code>cond</code>
l'ultima forma di <code>and</code> e di <code>or</code>	le forme precedenti
l'espansione di una macro	il corpo della macro che produce l'espansione
un dato valutato (<code>Cons</code> , dopo <code>data_to_form</code>)	gli argomenti di ogni chiamata
l'ultima forma di un gestore di <code>condition-case</code>	la forma protetta di <code>condition-case</code> , il corpo di <code>catch</code> e di <code>unwind-protect</code> , ogni forma di <code>while</code> , <code>dolist</code> , <code>dotimes</code> , <code>prog1</code> , <code>prog2</code>
una forma con una forma in testa, riscritta con il valore della testa	la testa stessa

Una ricorsione in coda non consuma stack: una funzione che si richiama in coda 200 000 volte termina normalmente. Una ricorsione *non* in coda consuma uno o più frame Rust per livello, e nessun contatore la limita: il carburante limita il numero di passi, non la profondità ($\rightarrow$ INT-2). `catch`, `condition-case` (sulla forma protetta) e `unwind-protect` valutano il corpo dentro il proprio frame, perché devono esserci quando l'errore risale, e quindi non sono trasparenti alla chiamata in coda.

6.2 Il permesso di sospendersi

`eval_step` esiste per un solo motivo: gestire il permesso di `yield`, una variabile per thread che dice se la valutazione che sta per cominciare è in una posizione da cui una fiber può essere sospesa e ripresa (Sezione 8).

da `eval.rs` — estratto semplificato

```
fn eval_step<T: LispContext>(exp: &LispExp<T>, env: Arc<Env<T>>, ctx: &T)
-> Result<EvalStep<T>, EvalError<T>>
{
    let permitted = take_yield_permission(); // lo prende e lo azzerava
    let step = eval_step_permitted(exp, env, ctx, permitted);
    // una chiamata in coda continua la stessa istruzione: il permesso passa
    if permitted && matches!(step, Ok(EvalStep::TailCall(..))) {
        grant_yield_permission();
    }
    step
}
```

Il permesso è *consumato* all'inizio di ogni passo: qualunque valutazione annidata dentro quel passo parte senza. È *restituito* quando il passo finisce con una chiamata in coda, perché la forma che continua è la stessa istruzione. Le posizioni che concedono il permesso sono poche e sono descritte nel Sezione 8.

6.3 Lo smistamento per tipo

`eval_step_permitted` addebita un'unità di carburante, prima di qualunque altra cosa, e poi guarda la variante dell'espressione.

```

ctx.consume_fuel(1)?;
match exp {
  String(_) | Number(_) | Atom(_) | Fiber(_) | Lambda(_) | Primitive { .. }
    => Ok(Done(exp.clone())),
  Symbol(s) => {
    if s == "nil" || s == "t" || s.starts_with('.') { return Ok(Done(exp.clone())); }
    env.get_variable(s).map(Done).ok_or(UnboundVariable(s.to_string()))
  }
  Cons(_) => Ok(TailCall(data_to_form(exp)?, env)),
  Form(list) if list.is_empty() => Ok(Done(LispExp::nil())),
  Form(list) => match &list[0] {
    Symbol(s) => eval_special_form_or_call_step(s, &list[1..], env, ctx, permitted),
    Form(_)   => /* valuta la testa, riscrivi la forma, TailCall */,
    Lambda(l) => /* chiama la chiusura direttamente */,
    _         => Err(InvalidFunctionCall),
  },
  Vector(v) => /* valuta ogni elemento, Done(nuovo vettore) */,
  Map(m)    => /* valuta ogni valore, Done(nuova mappa) */,
}

```

Variante	Valutazione
<code>String</code> , <code>Number</code> , <code>Atom</code> , <code>Fiber</code> , <code>Lambda</code> , <code>Primitive</code>	Valgono sé stessi.
<code>Symbol</code>	<code>nil</code> , <code>t</code> e le keyword (: nome) valgono sé stessi, senza cercarli; altrimenti si cerca la variabile, oppure <code>UnboundVariable</code> .
<code>Cons</code>	Un dato valutato: si converte con <code>data_to_form</code> e si restituisce come <code>TailCall</code> , così la forma ottenuta passa di nuovo da qui.
<code>Form</code> vuota	<code>nil</code> .
<code>Form</code> con testa simbolo	Forma speciale, macro o chiamata di funzione.
<code>Form</code> con testa <code>Form</code>	La testa viene valutata e la forma riscritta con il valore al suo posto: <code>((lambda (x) (* x 2)) 21)</code> vale 42.
<code>Form</code> con testa <code>Lambda</code>	Chiamata diretta della chiusura: argomenti valutati, nuovo ambiente, frame <code><lambda></code> nella pila delle chiamate. Accade quando la riscrittura della riga precedente mette una chiusura in testa.
<code>Form</code> con altra testa	<code>InvalidFunctionCall</code> : <code>(1 2 3)</code> non è una chiamata.
<code>Vector</code> , <code>Map</code>	Si valutano gli elementi o i valori; le chiavi di una mappa restano testo.

6.4 Le forme speciali

Quando la testa è un simbolo, `eval_special_form_or_call_step` confronta il nome con le forme speciali, scritte direttamente nei rami del suo `match`. Gli argomenti di una forma speciale arrivano *non valutati*: è la forma a decidere che cosa valutare, quando e in quale ambiente. La tabella è il riferimento completo; la

colonna «coda» dice quali sottoforme sono valutate come chiamata in coda, la colonna `yield` se il corpo può sospendersi.

Forma	Comportamento	Coda	yield
<code>quote</code>	Restituisce l'argomento, già convertito in dato dal lettore.	—	—
<code>backquote</code>	Modello con <code>,</code> (un valore) e <code>,@</code> (gli elementi di una lista). Un solo livello.	—	no
<code>if</code>	<code>(if C ALLORA [ALTRIMENTI])</code> . Solo <i>una</i> forma «altrimenti»: le successive sono ignorate.	sì	nei rami
<code>cond</code>	Clausole <code>(TEST CORPO...)</code> ; senza corpo restituisce il valore del test.	sì	sì
<code>and</code> , <code>or</code>	Cortocircuito; <code>(and)</code> è <code>t</code> , <code>(or)</code> è <code>nil</code> .	l'ultima	l'ultima
<code>when</code> , <code>unless</code>	Corpo valutato come un <code>progn</code> .	sì	sì
<code>progn</code>	Valuta il corpo, restituisce l'ultima forma.	sì	sì
<code>progl</code> , <code>prog2</code>	Restituiscono la prima o la seconda forma.	no	no
<code>while</code>	Ripete finché la condizione è vera. Restituisce il valore dell'ultima forma eseguita, <code>nil</code> se il corpo non gira mai.	no	sì
<code>dolist</code>	<code>(dolist (VAR LISTA [RISULTATO]) CORPO...)</code> .	no	no
<code>dotimes</code>	<code>(dotimes (VAR N [RISULTATO]) CORPO...)</code> , <code>VAR</code> da 0 a N-1.	no	no
<code>setq</code>	Coppie <code>NOME VALORE</code> ; restituisce l'ultimo valore.	no	—
<code>let</code> , <code>let*</code>	Legami in parallelo o in sequenza; <code>(let ())</code> è ammesso, <code>(let)</code> no.	sì	sì
<code>defvar</code> , <code>defconst</code>	<code>(defvar NOME [VALORE [DOC]])</code> ; restituisce il simbolo.	no	—
<code>defun</code>	<code>(defun NOME PARAMETRI [DOC] CORPO...)</code> .	—	—
<code>lambda</code>	Crea una chiusura sull'ambiente corrente; non registra docstring.	—	—
<code>defmacro</code>	Come <code>defun</code> , nello spazio delle macro; nessuna docstring.	—	—
<code>catch</code>	<code>(catch TAG CORPO...)</code> : intercetta i <code>throw</code> con lo stesso tag.	no	no
<code>condition-case</code>	<code>(condition-case VAR FORMA GESTORE...)</code> : intercetta gli errori.	nel gestore	no
<code>unwind-protect</code>	<code>(unwind-protect FORMA PULIZIA...)</code> : le pulizie girano sempre.	no	no
<code>spawn</code>	<code>(spawn (lambda () ...))</code> : esegue la chiusura su un nuovo thread.	—	—
<code>fiber</code>	<code>(fiber CORPO...)</code> : un programma sospendibile; <code>(fiber)</code> è già finito.	—	—
<code>yield</code>	<code>(yield [VALORE])</code> : sospende la fiber corrente.	—	—

`throw`, `signal`, `resume`, `funcall`, `apply` ed `eval` non sono forme speciali ma primitive: ricevono gli argomenti già valutati. Le sezioni seguenti descrivono gli algoritmi delle forme raggruppate per funzione;

`catch`, `condition-case` e `unwind-protect` sono nella Sezione 7, `fiber` e `yield` nella Sezione 8, `spawn` nella Sezione 10.

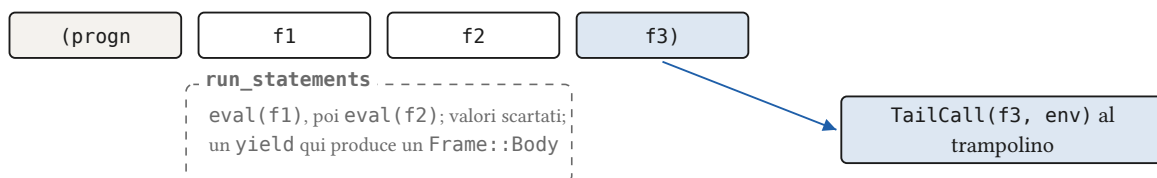
6.4.1 Corpi: `progn` e `eval_body_step`

Un *corpo* è una sequenza di forme di cui conta solo il valore dell'ultima. `progn`, `let`, `let*`, `when`, `unless`, le clausole di `cond` e il corpo di ogni funzione lo valutano con la stessa funzione:

da `eval.rs` – estratto semplificato

```
fn eval_body_step<T>(body: &[LispExp<T>], env: Arc<Env<T>>, ctx: &T, permitted: bool)
-> Result<EvalStep<T>, EvalError<T>>
{
    let Some((last, leading)) = body.split_last() else {
        return Ok(EvalStep::Done(LispExp::nil())); // corpo vuoto: nil
    };
    run_statements(&body[..leading.len()], 0, &env, ctx, permitted, LispExp::nil(),
        |from| Frame::Body { forms: body.to_vec(), from, env: env.clone() }?);
    Ok(EvalStep::TailCall(last.clone(), env)) // l'ultima: in coda
}
```

Le forme tranne l'ultima sono *istruzioni*: il loro valore si scarta. `run_statements` le valuta una a una, concedendo a ciascuna il permesso di sospendersi se il corpo lo aveva, e sa descrivere «il resto del corpo» se una di esse fa `yield` (il terzo argomento è la funzione che costruisce quella descrizione, Sezione 8). L'ultima forma non viene valutata qui: torna al trampolino come `TailCall`.



6.4.2 Condizionali: `if`, `cond`, `and`, `or`, `when`, `unless`

Tutti valutano le condizioni con `eval` ricorsiva e restituiscono come `TailCall` la forma da cui dipende il risultato:

- `if` valuta `C`; se è vera restituisce `TailCall(ALLORA)`, altrimenti `TailCall(ALTRIMENTI)` o, se manca, `nil`. Senza condizione è `IfNoConditionProvided`, senza ramo «allora» `IfNoTrueBranch`;
- `cond` prova le clausole in ordine; alla prima con il test vero valuta il corpo con `eval_body_step`, o restituisce il valore del test se il corpo è vuoto. Una clausola che non è una lista non vuota è `CondInvalidClause`;
- `and` valuta tutte le forme tranne l'ultima e si ferma con `nil` alla prima falsa; `or` si ferma con il primo valore vero; in entrambi i casi l'ultima forma è una chiamata in coda;
- `when` e `unless` valutano la condizione e poi il corpo con `eval_body_step`.

6.4.3 Cicli: `while`, `dolist`, `dotimes`

`while` valuta prima la condizione: se è falsa restituisce `nil` senza toccare il corpo. Poi le strade sono due. Se la forma *non* ha il permesso di sospendersi, gira un ciclo semplice: valuta le forme del corpo con `eval`, ricorda il valore dell'ultima, rivaluta la condizione. Se ha il permesso, passa a `run_while`, la stessa funzione che userà la ripresa di una *fiber* sospesa a metà di un'iterazione:

```
fn run_while<T>(condition: &LispExp<T>, forms: &[LispExp<T>], from: usize,
               env: &Arc<Env<T>>, ctx: &T, carried: LispExp<T>)
-> Result<LispExp<T>, EvalError<T>>
{
    let frame_at = |from| Frame::While { condition: ..., forms: ..., from, env: ... };
    // finisce l'iterazione in corso (da FROM), poi itera normalmente
    let mut last = run_statements(forms, from, env, ctx, true, carried, frame_at)?;
    loop {
        if eval(condition, env.clone(), ctx)?.is_nil() { return Ok(last); }
        last = run_statements(forms, 0, env, ctx, true, LispExp::nil(), frame_at)?;
    }
}
```

Tutte le forme del corpo di un `while` sono istruzioni, e quindi tutte possono sospendersi; la condizione no. `while` non crea un ambiente nuovo.

`dolist` e `dotimes` sono più semplici. Valutano la lista o il conteggio, creano un ambiente figlio con la variabile del ciclo, e per ogni elemento (o ogni intero da 0 a N-1) aggiornano la variabile con `update_variable` e valutano le forme del corpo con `eval`. La forma `RISULTATO`, se c'è, è valutata alla fine nell'ambiente del ciclo, dove la variabile vale ancora l'ultimo elemento (per `dolist`) o N-1. Il conteggio di `dotimes` è troncato a intero; una lista di `dolist` che non è una lista è `WrongArgumentType`. Il ciclo di questi due costrutti è scritto in Rust e consuma carburante solo attraverso le forme del corpo ($\rightarrow$ INT-5) nessuna delle loro forme può sospendersi.

6.4.4 Legami: `setq`, `let`, `let*`, `defvar`, `defconst`

`setq` richiede un numero pari di argomenti (`SetqWrongNumberOfArgs`) e un simbolo in ogni posizione dispari (`SetqSymbolRequired`). Per ogni coppia valuta il valore e prova `update_variable`; se nessun anello ha la variabile, la crea con `set_variable` nell'ambiente corrente. Le coppie sono assegnate in ordine, quindi `(setq a 1 b a)` dà a `b` il nuovo valore di `a`.

`let` crea un figlio, valuta *ogni* valore nell'ambiente *esterno* e lo lega nel figlio; poi valuta il corpo nel figlio. `let*` è identico, ma valuta ogni valore nel figlio stesso, dove i legami precedenti sono già visibili. Un legame è `(NOME FORMA)` oppure un `NOME` nudo, che vale `nil` (`parse_let_binding`); ogni altra forma è `LetUnvalidBindingAt(indice)`.

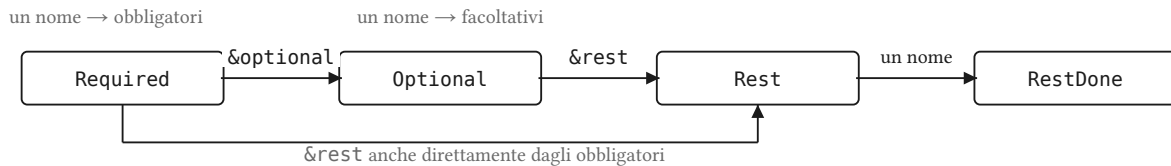
```
(let ((x 1)) (let ((x 2) (y x)) y)) ; => 1: y vede la x esterna
(let ((x 1)) (let* ((x 2) (y x)) y)) ; => 2: y vede la x appena legata
```

`defvar` e `defconst` controllano la forma prima di toccare qualunque cosa (al massimo tre argomenti, il primo un simbolo, la docstring una stringa), poi mettono la docstring nelle proprietà del simbolo a *ogni valutazione*, e infine assegnano: `defconst` sempre, `defvar` solo se `get_variable` non trova la variabile. È questa regola che permette di ricaricare un modulo senza perdere un valore che l'utente ha cambiato nel frattempo. Il valore è legato nell'ambiente corrente.

6.4.5 Definizioni: `defun`, `lambda`, `defmacro`

Le tre forme costruiscono una `Lambda` e differiscono solo in ciò che ne fanno: `lambda` la restituisce, `defun` la lega nello spazio delle funzioni, `defmacro` in quello delle macro. In tutti e tre i casi l'ambiente catturato è quello corrente.

La lista dei parametri è analizzata da `parse_lambda_params`, un piccolo automa a quattro stati che smista i simboli in tre gruppi:

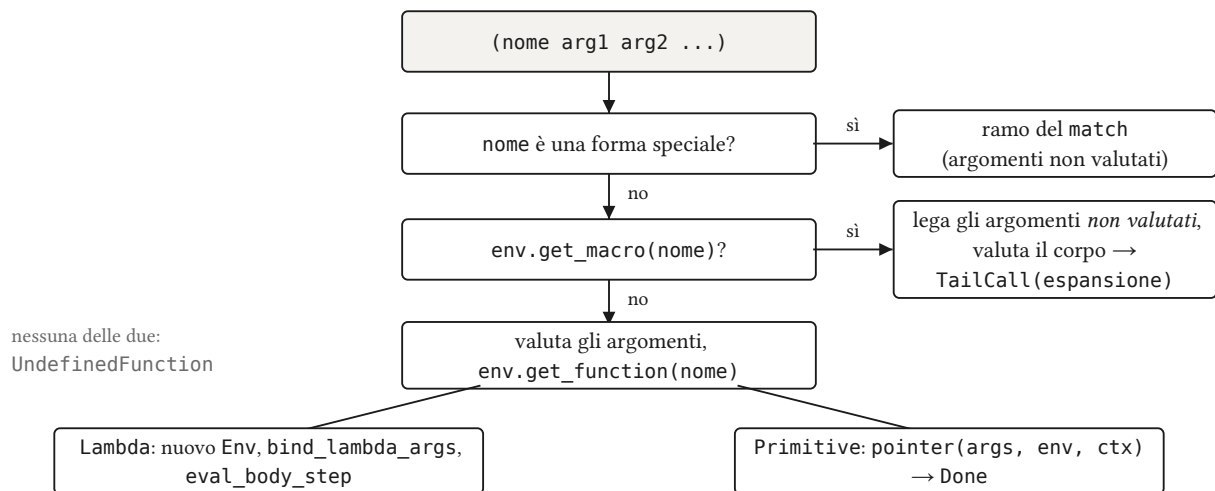


Un `&optional` fuori da `Required`, o un secondo `&rest`, è `DefunMisplacedParamMarker`; un `&rest` senza nome dopo, o con più di un nome, è `DefunRestMustHaveExactlyOneParam`; un parametro che non è un simbolo è `DefunParamIsNotASymbol`.

`defun` considera docstring il terzo argomento solo se è una stringa e dopo di esso ci sono altre forme: `(defun f () "ciao")` è una funzione che restituisce "ciao". `lambda` e `defmacro` non registrano docstring.

6.5 Chiamate di funzione e macro

Se il nome in testa non è una forma speciale, `eval_macro_or_function_call_step` cerca prima una macro e poi una funzione.



6.5.1 Chiamare una funzione

da `eval.rs` — estratto semplificato

```
fn eval_function_call_step<T>(symbol: &str, args: &[LispExp<T>], env: Arc<Env<T>>,
                             ctx: &T, permitted: bool) -> Result<EvalStep<T>, EvalError<T>>
{
    let mut evaled_args = Vec::new();
    for arg in args { evaled_args.push(eval(arg, env.clone(), ctx)?); } // da sinistra

    if let Some(func) = env.get_function(symbol) {
        ctx.push_call_frame(symbol);
        if let LispExp::Lambda(lambda) = func {
            let call_frame = Env::new_child(&lambda.env); // figlio della DEFINIZIONE
            bind_lambda_args(&lambda, &evaled_args, &call_frame)?;
            let step = eval_body_step(&lambda.body, call_frame, ctx, permitted);
            pop_unless_failed(ctx, &step);
            step
        } else if let LispExp::Primitive { pointer, .. } = func {
            let result = pointer(&evaled_args[..], env.clone(), ctx)?; // env del CHIAMANTE
            ctx.pop_call_frame();
            Ok(EvalStep::Done(result))
        } else { Err(EvalError::UncorrectFunctionDefinition) }
    } else { Err(EvalError::UndefinedFunction(symbol.into())) }
}
```

L'ordine conta. Gli argomenti sono valutati *prima* di cercare la funzione, ciascuno con una `eval` ricorsiva, quindi senza permesso di sospendersi. Il frame della pila delle chiamate è aperto prima di legare gli argomenti, così un numero sbagliato di argomenti compare nel backtrace con il nome della funzione chiamata. Una `Lambda` riceve un ambiente figlio del *suo* ambiente; una primitiva riceve l'ambiente del *chiamante*, ed è per questo che `eval`, `eval-string` e `boundp` vedono le variabili locali di chi le chiama. Il corpo della funzione non viene valutato qui: `eval_body_step` valuta le istruzioni e restituisce l'ultima forma come `TailCall`, e il frame viene tolto *prima* che il trampolino la valuti.

6.5.2 Legare gli argomenti: `bind_lambda_args`

da `utils.rs` — estratto semplificato

```
let min = lambda.params.len();
let max = min + lambda.optionals.len();
if args.len() < min || (lambda.rest.is_none() && args.len() > max) {
    return Err(WrongNumberOfArguments { expected: if args.len() < min { min } else { max },
                                           got: args.len() });
}
let mut idx = 0;
for name in &lambda.params {
    call_frame.set_variable(name.clone(), args[idx].clone());
    idx += 1;
}
for name in &lambda.optionals {
    let value = args.get(idx).cloned().unwrap_or_else(LispExp::nil);
    call_frame.set_variable(name.clone(), value);
    idx += 1;
}
if let Some(rest) = &lambda.rest {
    let start = idx.min(args.len());
    call_frame.set_variable(rest.clone(), LispExp::proper_list(args[start..].to_vec()));
}
```

Parametri	Argomenti	Legami
(a b)	1 2	a=1, b=2

Parametri	Argomenti	Legami
(a b)	1	WrongNumberOfArguments { expected: 2, got: 1 }
(a &optional b c)	1 2	a=1, b=2, c=nil
(a &optional b)	1 2 3	WrongNumberOfArguments { expected: 2, got: 3 }
(a &rest r)	1 2 3	a=1, r=(2 3)
(a &optional b &rest r)	1	a=1, b=nil, r=nil (l'indice è limitato con min)
(&rest r)	nessuno	r=nil

La stessa funzione lega gli argomenti di una macro. Lì gli «argomenti» sono le sottoforme non valutate, e il contenitore di `&rest` è comunque una lista di dati, che il corpo della macro scorre con `car` e `cdr`.

6.5.3 Espandere una macro

da `eval.rs` — estratto semplificato

```
if let Some(LispExp::Lambda(macro_lambda)) = env.get_macro(symbol) {
    let expand_frame = Env::new_child(&macro_lambda.env);
    bind_lambda_args(&macro_lambda, args, &expand_frame)?; // argomenti NON valutati
    let mut expansion = LispExp::nil();
    for form in &macro_lambda.body {
        expansion = eval(form, expand_frame.clone(), ctx)?; // il corpo produce un dato
    }
    Ok(EvalStep::TailCall(expansion, env)) // valutata dove sta la chiamata
}
```

Una macro è una funzione che riceve codice e restituisce codice. Il percorso di `(my-when t 1 2)` con la macro dell'esempio:

```
(defmacro my-when (c &rest body) `(if ,c (progn ,@body)))
(my-when t 1 2)
```

1. `get_macro("my-when")` trova la `Lambda`; si crea un figlio del suo ambiente.
2. `bind_lambda_args` lega `c` al simbolo `t` e `body` alla lista `(1 2)`: sono le sottoforme come le ha scritte il chiamante.
3. Il corpo, un backquote, produce il *dato* `(if t (progn 1 2))`, una catena di `Cons`.
4. Il dato torna al trampolino come `TailCall` nell'ambiente del *chiamante*.
5. Al giro successivo `eval_step_permitted` vede una `Cons`, la converte con `data_to_form` in una `Form` e la restituisce di nuovo come `TailCall`.
6. Il giro dopo valuta `(if t (progn 1 2))` come qualunque altra forma: vale `2`.

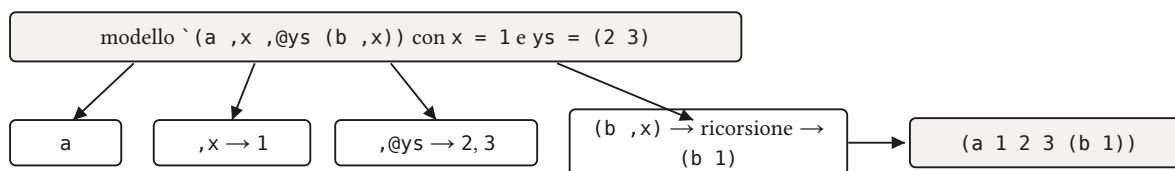
L'espansione avviene a ogni valutazione: non c'è una cache. Poiché è una chiamata in coda, eredita il permesso di sospendersi della posizione in cui compare la chiamata alla macro. Non esiste `gensym`: una macro che introduce variabili temporanee usa nomi che il codice dell'utente non userà (`save-match-data--saved` in `core/lisp/commands.lisp` ne è un esempio).

6.5.4 Il backquote

`eval_backquote` percorre il modello e costruisce un dato:

da `eval.rs` — estratto semplificato

```
match exp {
  LispExp::Form(list) => {
    if is_tagged(list, "unquote") { return eval(&list[1], env, ctx); } // ,X
    let mut result = Vec::new();
    for item in list.iter() {
      if item is Form(inner) && is_tagged(inner, "unquote-splicing") { // ,@X
        let spliced = eval(&inner[1], env.clone(), ctx?);
        // Cons o Form: gli elementi; nil: niente; altro: errore
        result.extend(elements of spliced);
        continue;
      }
      result.push(eval_backquote(item, env.clone(), ctx)?); // ricorsione
    }
    Ok(LispExp::proper_list(result))
  }
  LispExp::Vector(vec) => /* ogni elemento con eval_backquote, poi un vettore */,
  _ => Ok(exp.clone()), // atomi, Cons, mappe: invariati
}
```



il risultato è una lista di dati costruita con `proper_list`; `,@` aggiunge gli *elementi* della lista, `,` un solo valore

Un atomo, una `Cons` e una mappa sono restituiti come sono. Una lista puntata scritta nel modello è già una `Cons`, e quindi le sue virgole non vengono espanse; lo stesso vale per i valori di una mappa e per un backquote annidato ([→ INT-11](#)).

6.5.5 Chiamare da Rust: `call_callable`

da `base/mod.rs` — estratto semplificato

```
pub fn call_callable<T>(func: &LispExp<T>, call_args: &[LispExp<T>],
                        env: Arc<Env<T>>, ctx: &T) -> Result<LispExp<T>, EvalError<T>>
{
  match func {
    LispExp::Lambda(lambda) => {
      let call_frame = Env::new_child(&lambda.env);
      bind_lambda_args(lambda, call_args, &call_frame?);
      // ogni forma con eval diretta; l'ultima dà il valore
      for exp in &lambda.body[..last] { eval(exp, call_frame.clone(), ctx)?; }
      eval(lambda.body.last(), call_frame, ctx)
    }
    LispExp::Primitive { pointer: f, .. } => f(call_args, env, ctx),
    LispExp::Symbol(name) => match env.get_function(name) {
      Some(resolved) => call_callable(&resolved, call_args, env, ctx),
      None => Err(EvalError::UndefinedFunction(name.to_string())),
    },
    _ => Err(EvalError::UncoorectFunctionDefinition),
  }
}
```

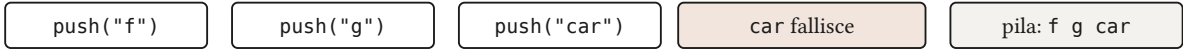
È il modo di chiamare qualcosa che si ha già in mano — una lambda, una primitiva o un simbolo che nomina una funzione — con argomenti già valutati. La usano `funcall`, `apply`, `mapcar`, `mapc` e `lisp` quando richiama Lisp da Rust (hook, callback del minibuffer, lavori di sfondo). Rispetto a una chiamata del valutatore

ha tre differenze: non registra un frame nel backtrace, non concede il permesso di sospendersi (un `yield` nel corpo è `YieldNotAllowed`), e l'ultima forma è valutata con una `eval` ricorsiva invece che in coda.

6.6 La pila delle chiamate per i backtrace

Prima del corpo di una funzione o di una primitiva il valutatore chiama `ctx.push_call_frame(nome)` (`<lambda>` per le chiusure chiamate in testa a una forma). La pila appartiene all'host; il valutatore si limita a dire quando un frame entra e quando esce, con un protocollo preciso implementato da `pop_unless_failed`:

Come finisce la chiamata	Cosa succede al frame
valore o chiamata in coda	<code>pop</code> : il lavoro del frame è finito.
errore	resta: quando l'errore arriva in cima, la pila descrive esattamente le chiamate attive al momento del guasto, ed è ciò che l'editor mostra come backtrace.
sospensione (<code>yield</code>)	<code>pop</code> : una fiber parcheggiata non sta chiamando nessuno.



```
(defun f () (g) nil) · (defun g () (car 5) nil) · (f)
```

Nessun frame viene tolto durante lo srotolamento: l'host trova in cima esattamente le tre chiamate attive.
Se lo stesso errore fosse intercettato, chi lo intercetta riporterebbe la pila alla profondità che aveva annotato.

tempo

Chi intercetta un errore (`catch`, `condition-case`, `unwind-protect`, `eval-string-safe`) annota `call_frame_depth()` prima e chiama `truncate_call_frames` dopo, così i frame di un fallimento già gestito non compaiono nel backtrace successivo.

Le chiamate in coda non appaiono nella pila: il frame di una funzione viene tolto quando il suo corpo restituisce l'ultima forma al trampolino, cioè *prima* che quella forma venga valutata. Nell'esempio le due funzioni terminano con `nil` proprio perché la chiamata che fallisce non sia l'ultima forma: con `(defun g () (car 5))` il frame di `g` sarebbe già uscito quando `car` fallisce, e la pila conterrebbe solo `f` e `car`.

7 Errori e uscite non locali

7.1 Un solo canale per tre cose diverse

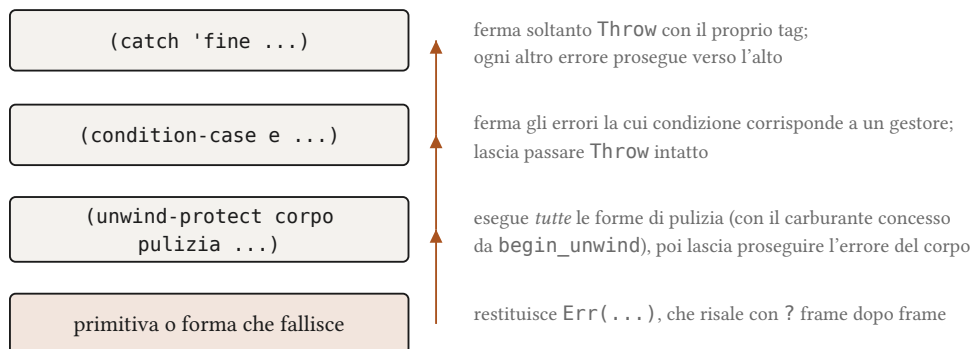
`EvalError<T>` è generico su `T` perché alcune varianti portano valori Lisp. Le varianti si dividono in tre famiglie.

Famiglia	Varianti	Chi le ferma
Errori	<code>UnboundVariable</code> , <code>UndefinedFunction</code> , <code>WrongNumberOfArguments</code> , <code>WrongArgumentType</code> , <code>OutOfFuel</code> , <code>RuntimeMessage</code> , <code>Signal { symbol, data }</code> , <code>YieldNotAllowed</code> e le forme malformate (tabella sotto)	<code>condition-case</code>
Trasferimenti	<code>Throw { tag, value }</code>	il <code>catch</code> con lo stesso tag
Sospensioni	<code>Yielded { value, frames }</code>	<code>resume</code>

Usare `Err` anche per `throw` e `yield` è la scelta naturale in Rust: `?` propaga già un `Err` attraverso ogni frame, quindi nessun punto del valutatore deve sapere che un trasferimento lo sta attraversando. Chi deve fermarne uno lo riconosce per variante.

Errore di forma malformata	Esempio che lo produce
<code>QuoteNotOneArgument</code>	<code>(quote a b)</code>
<code>IfNoConditionProvided</code> , <code>IfNoTrueBrach</code>	<code>(if)</code> , <code>(if t)</code>
<code>SetqSymbolRequired</code> , <code>SetqWrongNumberOfArgs</code>	<code>(setq 1 2)</code> , <code>(setq a)</code>
<code>DefunNameMustBeASymbol</code> , <code>DefunNotCorrectExpression</code>	<code>(defun "f" () 1)</code> , <code>(defun f ())</code>
<code>DefunParamsAreNotAList</code> , <code>DefunParamIsNotASymbol</code>	<code>(defun f x 1)</code> , <code>(lambda (1) 1)</code>
<code>DefunMisplacedParamMarker</code> , <code>DefunRestMustHaveExactlyOneParam</code>	<code>(lambda (&rest a &optional b) 1)</code> , <code>(lambda (&rest) 1)</code>
<code>LetNoBindingsProvided</code> , <code>LetUnvalidBindingList</code> , <code>LetUnvalidBindingAt(i)</code>	<code>(let)</code> , <code>(let 5 1)</code> , <code>(let ((1 2)) 1)</code>
<code>CondInvalidClause</code>	<code>(cond 5)</code> , <code>(cond ())</code>
<code>DolistInvalidBinding</code> , <code>DotimesInvalidBinding</code>	<code>(dolist x)</code> , <code>(dotimes (i))</code>
<code>DefvarNameMustBeASymbol</code> , <code>DefvarDocMustBeAString</code>	<code>(defvar)</code> , <code>(defvar x 1 2)</code>
<code>BackquoteNotOneArgument</code>	<code>(backquote a b)</code>
<code>ConditionCaseInvalidVariable</code> , <code>ConditionCaseInvalidHandler</code>	<code>(condition-case 5 x)</code> , <code>(condition-case e x 5)</code>
<code>UnvalidFunctionCall</code> , <code>UncorrectFunctionDefinition</code>	<code>(1 2)</code> , una lista puntata valutata; un valore non chiamabile nello spazio delle funzioni

7.2 Come si propaga



Un errore che nessuno ferma arriva a chi ha chiamato `eval` per primo: nel caso dell'editor, `run_command_form`, che lo mostra nell'area dei messaggi insieme al backtrace.

7.3 catch e throw

`throw` è una primitiva che restituisce `Err(Throw { tag, value })`. `catch` valuta il tag, annota la profondità della pila delle chiamate, poi valuta le forme del corpo una a una *dentro il proprio frame Rust*:

da `eval.rs` — estratto semplificato

```
let tag = eval(&args[0], env.clone(), ctx)?;
let depth_before = ctx.call_frame_depth();
let mut result = LispExp::nil();
for form in &args[1..] {
    match eval(form, env.clone(), ctx) {
        Ok(value) => result = value,
        Err(EvalError::Throw { tag: thrown, value }) => {
            if thrown == tag { // PartialEq: equal
                ctx.truncate_call_frames(depth_before);
                return Ok(EvalStep::Done(value));
            }
            return Err(EvalError::Throw { tag: thrown, value }); // non è il mio
        }
    }
    Err(err) => return Err(err),
}
Ok(EvalStep::Done(result))
```

Le forme non possono essere restituite al trampolino come `TailCall`: il trampolino le valuterrebbe *fuori* dal frame di `catch`, e il `throw` passerebbe oltre. Il tag si confronta con `PartialEq`, cioè strutturalmente: due stringhe uguali sono lo stesso tag. Un `throw` senza `catch` corrispondente arriva in cima come errore.

7.4 condition-case

da `eval.rs` – estratto semplificato

```
let depth_before = ctx.call_frame_depth();
let err = match eval(&args[1], env.clone(), ctx) {
    Ok(value) => return Ok(EvalStep::Done(value)),
    Err(throw @ EvalError::Throw { .. }) => return Err(throw), // non è un errore
    Err(err) => err,
};
let symbol = error_symbol(&err);
for handler in &args[2..] {
    let clause = /* (CONDIZIONE CORPO...) non vuota, o ConditionCaseInvalidHandler */;
    if !condition_matches(&clause[0], &symbol) { continue; }
    ctx.truncate_call_frames(depth_before);
    let handler_env = Env::new_child(&env);
    if let Some(name) = &var {
        let bound = LispExp::cons(symbol.clone(), error_data(&err));
        handler_env.set_variable(name.clone(), bound);
    }
    if clause.len() == 1 { return Ok(EvalStep::Done(LispExp::nil())); }
    for form in &clause[1..clause.len() - 1] { eval(form, handler_env.clone(), ctx)?; }
    return Ok(EvalStep::TailCall(clause[clause.len() - 1].clone(), handler_env));
}
Err(err) // nessun gestore: l'errore prosegue
```

La forma protetta è valutata nel frame di `condition-case`, per lo stesso motivo di `catch`. Un `Throw` passa intatto: è un trasferimento di controllo, non un fallimento. Per un errore vero, il nome della condizione viene da `error_symbol` e i gestori sono provati in ordine. Il primo che corrisponde riceve un ambiente figlio in cui `VAR` (se non è `nil`) vale `(SIMBOLO . DATI)`; il suo corpo è valutato come un corpo, con l'ultima forma in coda, perché a quel punto non c'è più nulla da proteggere.

EvalError	Condizione (error_symbol)	Dati (error_data)
Signal { symbol, data }	il simbolo stesso	data
UnboundVariable(n)	unbound-variable	(n)
UndefinedFunction(n)	undefined-function	(n)
UnvalidFunctionCall, UncorrectFunctionDefinition	invalid-function	la descrizione dell'errore
WrongNumberOfArguments	wrong-number-of-arguments	(atteso ricevuto)
WrongArgumentType	wrong-type-argument	("Tipo" valore)
OutOfFuel	out-of-fuel	la descrizione dell'errore
RuntimeMessage(m)	runtime-error	(m)
tutte le altre, compreso YieldNotAllowed	error	la descrizione dell'errore

`condition_matches` decide se una condizione copre il simbolo: `t` ed `error` coprono tutto, un simbolo copre sé stesso, una lista di condizioni copre ciò che copre uno qualunque dei suoi elementi. Non esistono gerarchie di condizioni (`define-error`): un gestore `error` intercetta anche i segnali con un simbolo inventato.

```
(condition-case e (car 5) (wrong-type-argument e))
;; => (wrong-type-argument "List" 5)
```

```
(condition-case nil (signal 'mio-errore '(1 2)) ((foo mio-errore) 'preso))
;; => preso
```

Intercettare `out-of-fuel` non serve a recuperare: quando l'errore arriva al gestore il carburante del thread è a zero, e il primo passo del gestore fallisce di nuovo con `OutOfFuel`. Il carburante si ricarica solo all'apertura del prossimo scope esterno (Sezione 9).

7.5 `unwind-protect`

da `eval.rs` — estratto semplificato

```
let body_result = eval(&args[0], env.clone(), ctx);
let depth_after_body = ctx.call_frame_depth();
ctx.begin_unwind(); // carburante per le pulizie
let mut cleanup_error = None;
for cleanup in &args[1..] {
    if let Err(err) = eval(cleanup, env.clone(), ctx) {
        if cleanup_error.is_none() { cleanup_error = Some(err); } // tiene il primo
    }
}
ctx.truncate_call_frames(depth_after_body);
match (body_result, cleanup_error) {
    (Err(body), Some(cleanup)) => { ctx.log_diagnostic(/* anche la pulizia */); Err(body) }
    (Err(body), None)          => Err(body),
    (Ok(_), Some(cleanup))    => Err(cleanup),
    (Ok(value), None)         => Ok(EvalStep::Done(value)),
}
```

Le pulizie girano *tutte*, anche se una di esse fallisce, e anche quando il corpo è uscito con un `throw` o con `OutOfFuel`. La tabella riassume l'esito:

Corpo	Pulizie	Risultato
valore	tutte riuscite	il valore del corpo
valore	una fallita	l'errore della prima pulizia fallita
errore o <code>throw</code>	tutte riuscite	l'errore del corpo, che prosegue
errore o <code>throw</code>	una fallita	l'errore del corpo; quello della pulizia va nella diagnostica

Prima delle pulizie il valutatore chiama `ctx.begin_unwind()`. Nell'editor questo alza il carburante rimasto ad almeno 100 000 passi, così un corpo uscito per `OutOfFuel` può ancora eseguire le sue pulizie; la chiamata avviene dopo ogni corpo, anche riuscito (→ INT-1).

7.6 `signal`, `eval-string-safe` e gli errori dell'host

`signal` è una primitiva che restituisce `Err(Signal { symbol, data })`: è il modo con cui il codice Lisp, e le primitive dell'host, sollevano un errore con un nome proprio. Una primitiva dell'editor che vuole un errore intercettabile come `invalid-regexp` restituisce proprio questa variante.

`eval-string-safe` è una barriera: legge e valuta la prima espressione della stringa, ripulisce la pila delle chiamate e restituisce sempre una lista, `(t VALORE)` o `(nil "descrizione dell'errore")`. Un errore del lettore diventa un `RuntimeMessage` con la descrizione del `ParserError`. È ciò che usa `M-`: nell'editor.

8 Fiber e `yield`

Una fiber è un programma che può fermarsi a metà, consegnare un valore a chi l'ha avviato e ripartire più tardi dallo stesso punto, con variabili e cicli come li aveva lasciati. È il meccanismo su cui l'editor costruisce i lavori di sfondo: un worker è una fiber che lo scheduler riprende un turno alla volta.

```
(setq f (fiber (let ((i 0))
  (while (< i 2)
    (setq i (+ i 1))
    (yield i))
  'fine)))
(resume f) ; => 1
(resume f) ; => 2
(resume f) ; => fine e ora (fiber-done-p f) è t
(resume f) ; => nil
```

8.1 Il problema: lo stack di Rust non si può conservare

Il valutatore è una funzione Rust ricorsiva: mentre valuta il `yield` dentro il `while` dentro il `let`, il punto in cui il programma si trova è lo stack di Rust, con un frame per ciascuna di quelle forme. Per consegnare il valore a chi ha chiamato `resume` bisogna tornare a quel chiamante, cioè srotolare lo stack, e srotolarlo lo distrugge.

La soluzione è che, durante lo srotolamento, ogni blocco attraversato scriva *che cosa gli resta da fare* in una struttura dati, un `Frame`. Alla ripresa non si ricostruisce lo stack: si esegue quella descrizione. Questo funziona solo nei punti in cui «ciò che resta da fare» è descrivibile semplicemente, e il meccanismo dei permessi serve a garantire che un `yield` avvenga solo lì.

8.2 La rappresentazione

da `types.rs` — estratto semplificato

```
pub enum Frame<T: LispContext> {
  // il resto di un corpo: le forme da FROM in poi, nell'ambiente ENV
  Body { forms: Vec<LispExp<T>>, from: usize, env: Arc<Env<T>> },
  // il resto di un'iterazione di un while, poi il ciclo stesso
  While { condition: LispExp<T>, forms: Vec<LispExp<T>>, from: usize, env: Arc<Env<T>> },
}
pub struct FiberState<T: LispContext> {
  pub pending: Vec<Frame<T>>, // il blocco più interno per primo
  pub is_done: bool,
}
pub struct SharedFiber<T: LispContext>(pub Arc<RwLock<FiberState<T>>>);
```

Una fiber appena creata da `(fiber CORPO...)` è una pila con un solo `Frame::Body` che contiene tutto il corpo, da `from = 0`, in un ambiente figlio di quello in cui la fiber è stata creata: non esiste un percorso separato per «la prima esecuzione», che è semplicemente una ripresa da prima della prima forma. `(fiber)` senza corpo nasce già finita.

8.3 Dove si può sospendere

Un'istruzione di un corpo e un'istruzione del corpo di un `while` sono i soli punti in cui si sa descrivere il seguito: le istruzioni successive, e nel caso del `while` il ciclo stesso. Il permesso di sospendersi è una variabile del thread, `YIELD_PERMITTED`:

- lo concede `run_statements`, prima di ogni istruzione, se il blocco stesso lo aveva; lo concede in ogni caso quando esegue un frame durante una ripresa;

- lo *consuma* `eval_step` all'inizio di ogni passo, così qualunque valutazione annidata parte senza;
- lo *restituisce* `eval_step` quando il passo finisce con una chiamata in coda, perché la forma che continua è la stessa istruzione.

Il `yield` controlla il permesso *prima* di valutare il proprio argomento:

da `eval.rs` — estratto semplificato

```
"yield" => {
  if !permitted { return Err(EvalError::YieldNotAllowed); }
  let value = match args.first() {
    Some(form) => eval(form, env, ctx)?, // un argomento: niente permesso
    None => LispExp::nil(),
  };
  Err(EvalError::Yielded { value, frames: Vec::new() })
}
```

Ne discende la tabella delle posizioni, verificata eseguendo l'interprete:

Posizione del <code>yield</code>	Esito
istruzione di <code>progn</code> , <code>let</code> , <code>let*</code> , <code>when</code> , <code>unless</code> , <code>cond</code> , corpo di funzione chiamata per nome	sospende
istruzione del corpo di <code>while</code> (quando il <code>while</code> stesso è in una posizione permessa)	sospende
ramo di <code>if</code> , ultima forma di <code>and</code> / <code>or</code> , espansione di una macro (chiamate in coda)	sospende
argomento di una chiamata, condizione, valore di un <code>let</code>	<code>YieldNotAllowed</code>
<code>dolist</code> , <code>dotimes</code> , <code>progl</code> , <code>prog2</code>	<code>YieldNotAllowed</code>
<code>catch</code> , <code>unwind-protect</code> , <code>condition-case</code>	<code>YieldNotAllowed</code> , che è un errore come gli altri: un gestore <code>error</code> di <code>condition-case</code> lo intercetta (→ INT-10)
chiusura chiamata con <code>funcall</code> , <code>mapc</code> , <code>mapcar</code> (<code>call_callable</code>)	<code>YieldNotAllowed</code>
fuori da una fiber	<code>YieldNotAllowed</code>

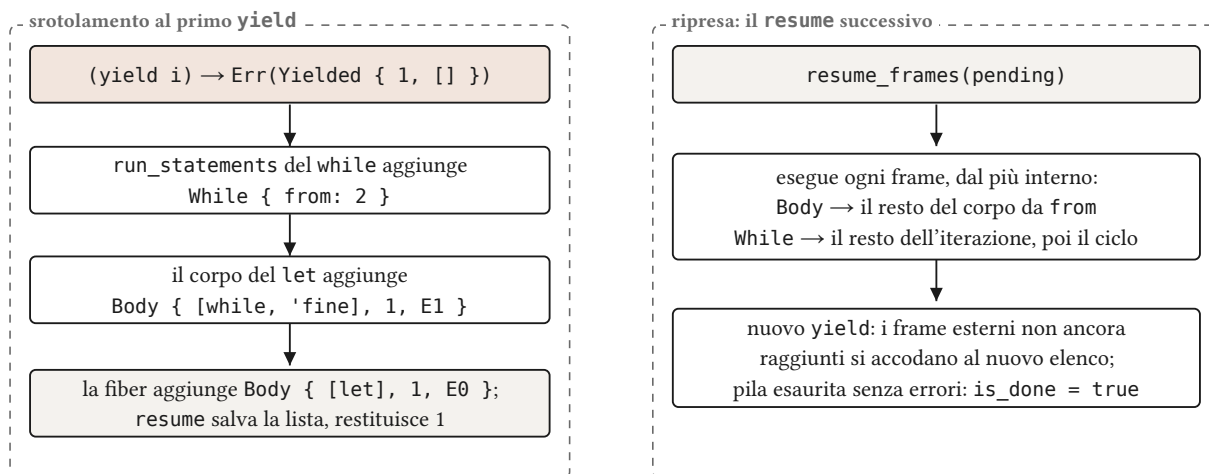
8.4 Lo srotolamento: chi scrive i frame

`run_statements` è il cuore del meccanismo: valuta le istruzioni e, se una di esse restituisce `Yielded`, aggiunge alla lista dei frame la descrizione del proprio seguito e lascia proseguire l'errore.

da `eval.rs` – estratto semplificato

```
fn run_statements<T, F>(forms: &[LispExp<T>], from: usize, env: &Arc<Env<T>>, ctx: &T,
    permitted: bool, carried: LispExp<T>, frame_at: F)
-> Result<LispExp<T>, EvalError<T>>
where F: Fn(usize) -> Frame<T>
{
    let mut last = carried;
    for (index, form) in forms.iter().enumerate().skip(from) {
        if permitted { grant_yield_permission(); }
        match eval(form, env.clone(), ctx) {
            Ok(value) => last = value,
            Err(EvalError::Yielded { value, mut frames }) => {
                frames.push(frame_at(index + 1)); // riprendi dalla successiva
                return Err(EvalError::Yielded { value, frames });
            }
            Err(other) => return Err(other),
        }
    }
    Ok(last)
}
```

Poiché ogni blocco aggiunge il proprio frame *dopo* quelli dei blocchi più interni, la lista finisce ordinata dal più interno al più esterno. Il frame punta all'istruzione *successiva* (`index + 1`): quella che ha fatto `yield` è considerata finita, e il suo valore è il valore del `yield`.



8.5 La ripresa: `resume` e `resume_frames`

da `base/fibers.rs` – estratto semplificato

```
let frames = {
    let mut fiber = shared_fiber.0.write()?;
    if fiber.is_done { return Ok(LispExp::nil()); }
    std::mem::take(&mut fiber.pending) // la pila esce dalla fiber...
}; // ...e il lock è rilasciato qui
if frames.is_empty() { /* is_done = true */ return Ok(LispExp::nil()); }
let outcome = resume_frames(frames, ctx); // gira senza lock
let mut fiber = shared_fiber.0.write()?;
match outcome {
    Err(EvalError::Yielded { value, frames }) => { fiber.pending = frames; Ok(value) }
    Ok(value) => { fiber.is_done = true; Ok(value) }
    Err(other) => { fiber.is_done = true; fiber.pending.clear(); Err(other) }
}
```

`resume` toglie la pila dalla fiber e rilascia il lock prima di eseguirla: il codice della fiber può quindi chiedere `fiber-done-p` di sé stessa senza stallo. Un secondo `resume` che arriva mentre la fiber gira trova una pila vuota e la segna come finita ($\rightarrow$ INT-9). Un errore dentro una fiber la termina: il punto di ripresa è andato perduto con lo srotolamento, e riprendere vorrebbe dire ricominciare da un punto già superato.

da `eval.rs` — estratto semplificato

```
pub fn resume_frames<T>(frames: Vec<Frame<T>>, ctx: &T) -> Result<LispExp<T>, EvalError<T>> {
    let mut last = LispExp::nil();
    let mut outer = frames.into_iter();
    while let Some(frame) = outer.next() {
        let finished = match frame {
            Frame::Body { forms, from, env } =>
                run_statements(&forms, from, &env, ctx, true, last.clone(), /* ... */),
            Frame::While { condition, forms, from, env } =>
                run_while(&condition, &forms, from, &env, ctx, last.clone()),
        };
        match finished {
            Ok(value) => last = value,
            Err(EvalError::Yielded { value, mut frames }) => {
                frames.extend(outer); // i frame esterni non ancora raggiunti
                return Err(EvalError::Yielded { value, frames });
            }
            Err(other) => return Err(other),
        }
    }
    Ok(last)
}
```

Due dettagli rendono la ripresa equivalente all'esecuzione senza interruzioni:

- il valore di un blocco che non ha più istruzioni da eseguire è il valore prodotto dai blocchi al suo interno: per questo ogni frame riceve `last` come valore *portato* (*carried*). Senza, `(fiber (progn (yield 1) 7))` risponderebbe `nil` alla seconda ripresa invece di `7`;
- se la ripresa di un blocco si sospende di nuovo, i frame esterni non ancora eseguiti vengono accodati alla nuova lista: sono ancora la strada per tornare indietro.

8.6 L'esempio, frame per frame

La tabella segue la fiber dell'inizio del capitolo. E0 è l'ambiente della fiber, E1 quello del `let`.

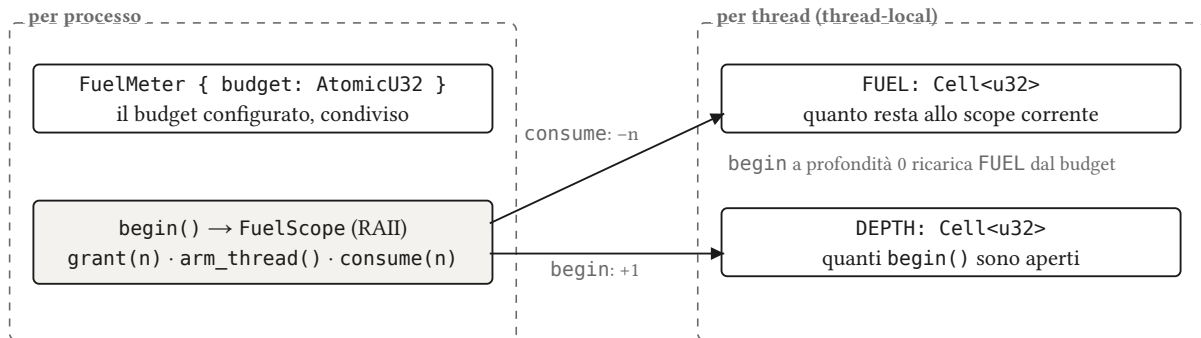
resume	Valore	pending dopo la chiamata
1°	1	While { from: 2 } · Body { [while, 'fine], from: 1, E1 } · Body { [let], from: 1, E0 }
2°	2	Il While finisce l'iterazione (niente dopo il <code>yield</code>), rivaluta la condizione, rientra e si sospende di nuovo: While { from: 2 } più i due Body accodati, invariati.
3°	fine	Il While trova la condizione falsa; il primo Body esegue 'fine dall'indice 1; il secondo non ha più nulla e restituisce il valore portato, fine. Pila vuota: <code>is_done = true</code> .
4°	nil	La fiber è finita: <code>resume</code> restituisce <code>nil</code> senza eseguire nulla.

`fiber-done-p` è l'unico modo affidabile di distinguere una fiber finita da una che ha fatto `yield` di `nil`.

9 Il carburante

Il carburante protegge dai cicli infiniti. Mentre un comando gira, l'editor non legge la tastiera, quindi un `(while t ...)` scritto per sbaglio non potrebbe essere interrotto: il carburante lo ferma dopo un numero stabilito di passi con l'errore `OutOfFuel`. L'interprete fornisce il *meccanismo* in `fuel.rs`; quanto carburante dare e quando ricaricarlo è *politica* dell'host.

9.1 Il contatore e il budget



Il budget configurato è uno solo per processo, in un `AtomicU32` dentro il `FuelMeter`. Il *residuo* è invece una variabile per thread, per due motivi: nessuna sincronizzazione sul percorso caldo (lo si tocca a ogni passo), e un thread creato da `spawn` non può spendere il budget del thread che lo ha creato, né vederselo ricaricare sotto i piedi.

9.2 Le operazioni

Operazione	Effetto
<code>consume(n)</code>	Toglie <code>n</code> dal residuo; se non basta restituisce <code>Exhausted</code> , che l'host converte in <code>EvalError::OutOfFuel</code> . Il residuo non scende mai sotto zero.
<code>begin() → FuelScope</code>	Apri uno scope: se <code>DEPTH</code> passa da 0 a 1 ricarica il residuo dal budget, altrimenti aumenta solo <code>DEPTH</code> . La guardia restituita chiude lo scope quando viene distrutta, anche in caso di <code>?</code> o di ritorno anticipato.
<code>grant(n)</code>	Alza il residuo ad <i>almeno</i> <code>n</code> , senza abbassarlo mai. Serve alle pulizie di <code>unwind-protect</code> .
<code>arm_thread()</code>	Carica il budget nel residuo di un thread appena creato, che non ha uno scope in cui annidarsi.
<code>set_remaining(n)</code>	Fissa il residuo esattamente a <code>n</code> , alzandolo o abbassandolo: serve ai turni brevi dei worker dell'editor, ciascuno con la propria piccola quota.
<code>measure(meter, f)</code> (solo test)	Esegue <code>f</code> con un residuo praticamente illimitato e riporta quante unità ha speso: un numero esatto e riproducibile su ogni macchina, adatto a fissare regressioni nei test.

Un thread che non apre mai uno scope parte comunque da `DEFAULT_FUEL` (10 000 000 di passi, circa un secondo in release): è limitato anche senza la collaborazione dell'host. Il numero è una protezione contro le fughe, non una quota: il lavoro normale ne usa una frazione minima.

9.3 Dove si paga

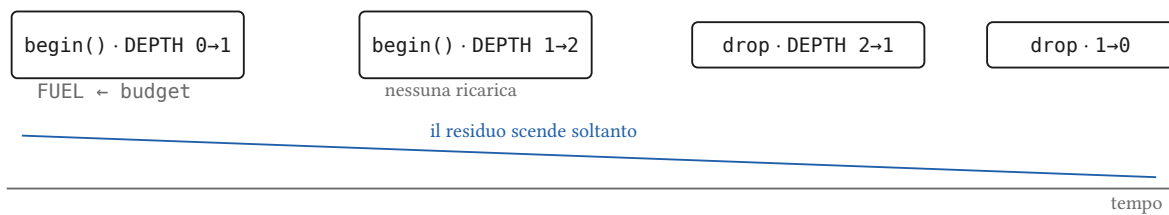
Chi addebita	Quanto
<code>eval_step_permitted</code>	un'unità all'ingresso di ogni passo, prima di sapere che cosa farà
<code>expect_list</code> (primitive di base che scorrono liste)	un'unità per elemento: <code>length</code> , <code>mapcar</code> , <code>member</code> su una lista enorme si fermano in tempo
primitive dell'editor	alcune addebitano ciò che scorrono (testo, percorsi, candidati di completamento)

Ciò che gira in Rust senza passare da questi punti non paga: il ciclo di `dotimes` con un corpo vuoto, l'allocazione di `make-string`, il lavoro interno di una primitiva (→ INT-5).

Qualche conto, per farsi un'idea delle unità: `(+ 1 (* 2 3))` costa 5 unità (la forma esterna, `1`, la forma interna, `2`, `3`); un giro di `(while (< i n) (setq i (+ i 1)))` ne costa 7 (la condizione e i suoi due argomenti, il `setq`, il `+` e i suoi due argomenti), quindi il budget predefinito basta per circa un milione e mezzo di giri.

9.4 Il residuo durante un comando

La figura segue il residuo durante un comando dell'editor che ne valuta un altro al proprio interno (per esempio `eval-file` dentro un comando).



Ricaricare solo al passaggio da 0 a 1 è ciò che impedisce a un programma di procurarsi un budget nuovo rientrando nel valutatore a metà di un comando.

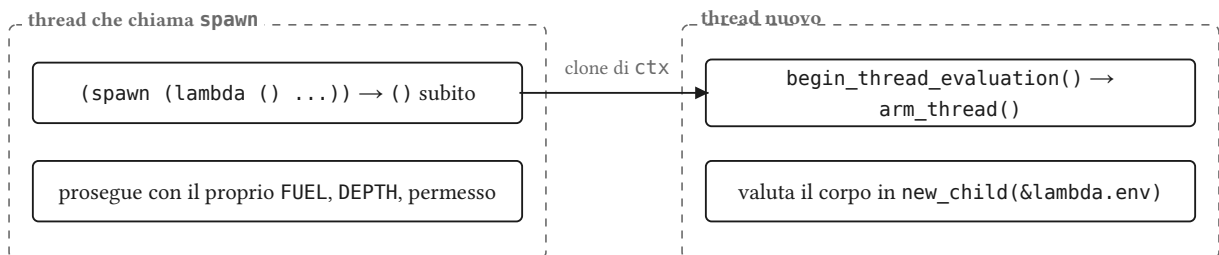
10 Thread e stato condiviso

`(spawn (lambda () ...))` avvia un thread del sistema operativo che valuta le forme del corpo della chiusura.

da `eval.rs` – estratto semplificato

```
let target_closure = eval(&args[0], env.clone(), ctx)?;
if let LispExp::Lambda(lambda_data) = target_closure {
    let lambda_clone = lambda_data.clone();
    let mut thread_ctx = ctx.clone();
    std::thread::spawn(move || {
        thread_ctx.begin_thread_evaluation(); // arma il carburante
        let thread_frame = Env::new_child(&lambda_clone.env);
        for exp in &lambda_clone.body {
            if let Err(err) = eval(exp, thread_frame.clone(), &mut thread_ctx) {
                thread_ctx.log_diagnostic(&format!("[LISP thread] {err:?}"));
                break; // nessuno a cui restituirlo
            }
        }
    });
}
Ok(EvalStep::Done(LispExp::form(vec![]))) // subito, una lista vuota
}
```

Il nuovo thread riceve un *clone* del contesto e un ambiente figlio di quello della chiusura. I parametri della lambda non vengono legati: il corpo vede soltanto ciò che la chiusura ha catturato. Il primo errore interrompe il thread e finisce nella diagnostica, perché non c'è nessuno a cui restituirlo. `spawn` restituisce subito una forma vuota, che si stampa `()` (→ INT-18).



condiviso tra i due thread: gli ambienti (ogni tabella ha il suo `RwLOCK`), gli atomi, le fiber, l'obarray.
Separato: carburante, profondità degli scope, permesso di `yield`.

Stato per thread	Perché è separato
FUEL, DEPTH	un budget appartiene a una sola linea di esecuzione
YIELD_PERMITTED	il permesso di un'istruzione su un thread non deve autorizzare un argomento su un altro
lato editor: dati di <code>string-match</code> , cache delle espressioni regolari, contrassegno dei comandi per l'annullamento	ognuno descrive ciò che sta facendo <i>quel</i> thread

Gli atomi (`make-atom`, `deref`, `reset`) sono il modo previsto di scambiare valori mutabili tra thread: `deref` prende il lock di lettura, `reset` quello di scrittura. Non esiste un aggiornamento atomico «leggi, calcola, scrivi» (→ INT-21). Due thread che fanno `setq` sulla stessa variabile globale non si danneggiano (ogni tabella ha il suo lock), ma non hanno alcuna garanzia d'ordine.

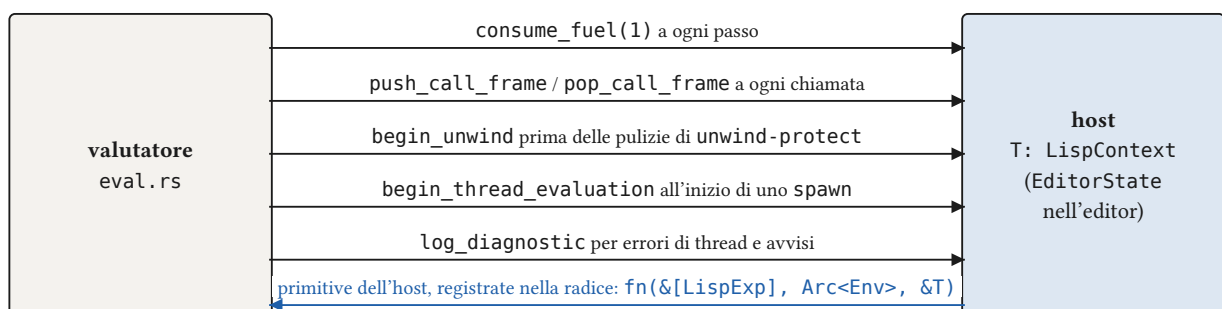
11 Il contratto con l'host

11.1 Il trait `LispContext`

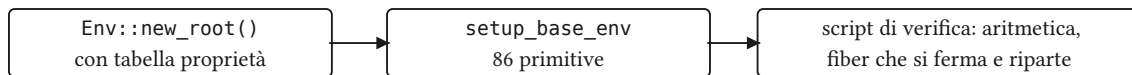
```
pub trait LispContext: Clone + PartialEq + Debug + Send + Sync + 'static {  
    fn consume_fuel(&self, amount: u32) -> Result<(), EvalError<Self>> { Ok(()) }  
    fn log_diagnostic(&self, msg: &str) {}  
    fn begin_unwind(&self) {}  
    fn begin_thread_evaluation(&self) {}  
    fn push_call_frame(&self, frame: &str) {}  
    fn pop_call_frame(&self) {}  
    fn call_frame_depth(&self) -> usize { 0 }  
    fn truncate_call_frames(&self, depth: usize) {}  
}
```

Ogni metodo ha un'implementazione vuota, così un host che non misura, non registra e non tiene backtrace non paga nulla: i test dell'interprete usano `T = ()`. Il contesto è passato per riferimento a ogni `eval` e a ogni primitiva, ed è clonato soltanto da `spawn`; i vincoli `Send + Sync + 'static` esistono proprio per `spawn`.

Metodo	Quando lo chiama l'interprete	Che cosa fa l'editor
<code>consume_fuel(n)</code>	a ogni passo (<code>n = 1</code>) e dalle primitive che scorrono liste	scala il <code>FuelMeter</code> del compartimento <code>Runtime</code> ; <code>OutOfFuel</code> se non basta
<code>log_diagnostic</code>	errori nei thread di <code>spawn</code> , pulizie fallite, avvisi delle primitive	scrive nel registro dell'editor (e nel file, se è attivo)
<code>begin_unwind</code>	prima delle pulizie di <code>unwind-protect</code>	<code>grant(100 000)</code>
<code>begin_thread_evaluation</code>	all'inizio di uno <code>spawn</code>	<code>arm_thread()</code>
<code>push_call_frame</code> , <code>pop_call_frame</code>	all'ingresso e all'uscita di una chiamata (Sezione 6.6)	una <code>Vec<String></code> nel compartimento <code>Runtime</code>
<code>call_frame_depth</code> , <code>truncate_call_frames</code>	<code>catch</code> , <code>condition-case</code> , <code>unwind-protect</code> , <code>eval-string-safe</code>	legge e accorcia la stessa <code>Vec</code>



11.2 L'avvio: `bootstrap_vm`



se lo script non produce `vm-status-healthy`, `bootstrap_vm` restituisce un errore: un interprete rotto
si scopre all'avvio, non al primo tasto. L'host aggiunge poi le proprie primitive alla stessa radice.

Lo script calcola `(- (+ 5 5) 2)` e fa partire una fiber `(fiber (yield 100) 200)`, che al primo `resume` deve restituire 100: una fiber che non si fermasse darebbe 200, una che non partisse darebbe `nil`. Il risultato atteso è il simbolo `vm-status-healthy`; altrimenti `bootstrap_vm` scrive il risultato nella diagnostica e restituisce `UncorrectFunctionDefinition`. Lo script usa `setq` nella radice, quindi le tre variabili che usa restano globali (→ INT-19).

11.3 Cosa l'interprete esporta

`lisp/mod.rs` rende pubblici, all'interno del crate: `eval`, `resume_frames`, `Parser`, `form_to_data`, `Env`, `VARIABLE_DOCUMENTATION`, `LispExp`, `Lambda`, `Frame`, `LispPrimitive`, `EvalError`, `LispContext`, `FuelMeter`, `FuelScope`, `DEFAULT_FUEL`, `set_remaining`, `bootstrap_vm`, `setup_base_env`, `call_callable`, `lisp_display`, `exact_arity`, `some_arguments` (e `measure` nei test). Il crate `rsedit_core` riesporta verso l'esterno soltanto `Env`, `LispContext`, `Parser` ed `eval`.

12 Le primitive di base

12.1 Organizzazione

Una primitiva sta in `base/` se avrebbe lo stesso significato in un Lisp senza editor: il criterio è poterla scrivere contro `T: LispContext` senza sapere che cosa sia `T`. Ogni modulo ha una funzione `install(into: &Registry<T>)` che registra le proprie primitive; `setup_base_env` è soltanto l'elenco dei moduli, in ordine alfabetico.

da `base/mod.rs` — estratto semplificato

```
pub(crate) struct Registry<'a, T: LispContext> { env: &'a Arc<Env<T>> }
impl<T: LispContext> Registry<'_, T> {
    pub(crate) fn function(&self, name: &str, pointer: LispPrimitive<T>, doc: &'static str) {
        self.env.set_function(name.into(), LispExp::primitive(pointer, Some(doc.into())));
    }
}
```

Modulo	N.	Primitive
atoms	3	make-atom, deref, reset
comparisons	7	=, <, >, <=, >=, max, min
fibers	2	resume, fiber-done-p
functions	8	funcall, eval, eval-string, eval-string-safe, throw, signal, function-doc, variable-doc
inspect	6	boundp, symbol-value, function-arglist, all-functions, all-macros, all-variables
lists	21	car, cdr, cons, list, nth, nthcdr, length, append, reverse, member, memq, assoc, assq, elt, mapcar, mapc, apply, identity, add-to-list, append-to-list, remove-from-list
math	10	+, -, *, /, mod, %, abs, floor, 1+, 1-
predicates	14	eq, eql, equal, null, not, consp, listp, stringp, numberp, symbolp, functionp, vectorp, zerop, atom
strings	13	concat, substring, string=, string<, upcase, downcase, format, make-string, number-to-string, string-to-number, symbol-name, intern, split-string
symbols	2	put, get

12.2 Gli strumenti comuni

`base/mod.rs` contiene ciò che i moduli hanno in comune. Le macro generano le primitive ripetitive, così una famiglia intera è una riga per primitiva:

Macro	Che primitiva genera
predicate!	Un argomento; <code>t</code> se corrisponde a un pattern (o, con <code>not</code> , se non corrisponde). <code>consp</code> , <code>stringp</code> , <code>atom</code> e simili.
number_op!, string_op!	Un argomento numerico o stringa a cui si applica un metodo di <code>f64</code> o di <code>String</code> (<code>abs</code> , <code>floor</code> , <code>upcase</code>).
fold_numbers!	Almeno un argomento; piega i numeri con un metodo (<code>max</code> , <code>min</code>).
name_list!	Nessun argomento; la lista ordinata dei nomi di uno spazio (<code>all-functions</code>).
raise!	Due argomenti; restituisce un <code>Err</code> con quei due campi (<code>throw</code> , <code>signal</code>).
nil!	Abbreviazione di <code>LispExp::nil()</code> .

Le funzioni di aiuto leggono gli argomenti in modo uniforme:

- `expect_number` ed `expect_string` restituiscono il valore o `WrongArgumentType` con il nome del tipo atteso;
- `expect_list` accetta una lista o `nil`, ne copia gli elementi in un vettore e *addebita un'unità di carburante per elemento* (`charge`);
- `call_callable` chiama una lambda, una primitiva o un simbolo (Sezione 6.5);
- `find_member` e `find_assoc` sono il cuore di `member / memq` e di `assoc / assq`: scorrono la lista con `expect_list` e confrontano con `PartialEq`, cioè con `equal` (→ INT-16)
- `compare_chain` controlla una relazione su ogni coppia adiacente di numeri: `(< 1 2 3)` è vero se `1 < 2` e `2 < 3`.

12.3 Semantiche da conoscere

Primitive	Comportamento
<code>+ - * /</code>	Tutto in <code>f64</code> : <code>(/ 1 3)</code> vale <code>0.333...</code> ; per una divisione intera si usa <code>floor</code> . <code>(- x)</code> cambia segno, <code>(/ x)</code> è il reciproco; dividere per zero è un <code>RuntimeMessage</code> , non un infinito.
<code>mod , %</code>	La stessa funzione: il segno del risultato segue il divisore, <code>(% -7 2)</code> vale 1.
<code>car , cdr</code>	Su <code>nil</code> restituiscono <code>nil</code> ; su una lista puntata <code>cdr</code> restituisce la coda.
<code>nth , elt</code>	Indice fuori dai limiti o negativo: <code>nil</code> . <code>elt</code> funziona anche su vettori e stringhe.
<code>length , reverse</code>	Liste, vettori e stringhe (in caratteri).
<code>append</code>	Concatena; l'ultimo argomento diventa la coda così com'è.
<code>mapcar , mapc , apply</code>	Accettano una lambda, una primitiva o un simbolo che nomina una funzione; chiamano con <code>call_callable</code> .
<code>substring</code>	Indici in caratteri, negativi contati dalla fine.
<code>format</code>	<code>%s / %S</code> (come <code>lisp_display</code>), <code>%d</code> (tronca a intero), <code>%f</code> , <code>%%</code> ; meno argomenti del necessario è un errore. Riconosce anche le sequenze <code>\n</code> , <code>\t</code> , <code>\r</code> scritte nella stringa di formato e rifiuta le altre (→ INT-12).
<code>make-string</code>	<code>(make-string N "x")</code> ripete il <i>primo carattere</i> della stringa N volte.
<code>string-to-number</code>	0 per un testo non numerico; usa <code>f64::from_str</code> .
<code>split-string</code>	Senza separatori divide sugli spazi bianchi e scarta i pezzi vuoti; con una stringa vuota divide in caratteri; altrimenti divide sulle occorrenze letterali della stringa data (<code>"::"</code> è un solo separatore di due caratteri, non due separatori).
<code>intern , symbol-name</code>	Da stringa a simbolo e ritorno.
<code>eval , eval-string</code>	Valutano nell'ambiente del <i>chiamante</i> : <code>(let ((x 5)) (eval 'x))</code> vale 5. <code>eval-string</code> valuta solo la prima espressione.
<code>boundp , symbol-value</code>	Guardano lo spazio delle variabili dall'ambiente del chiamante.
<code>function-arglist</code>	La lista dei parametri come stringhe, con <code>&optional</code> e <code>&rest</code> .
<code>put , get</code>	La tabella delle proprietà nella radice (Sezione 5.6).
<code>add-to-list , append-to-list , remove-from-list</code>	Ricevono il <i>simbolo</i> della lista, calcolano una nuova lista e la legano con <code>set_variable</code> nell'ambiente del chiamante.

12.4 Ricetta: aggiungere una primitiva di base

1. Scegli il modulo per argomento. Se nessuno va bene, crea un file, dichiaralo in `base/mod.rs` e aggiungi la sua `install` a `setup_base_env`.
2. Scrivi la docstring come `const`. La prima riga *deve* essere la firma seguita da una frase, nella forma `(nome ARG &optional ALTRO): cosa fa.: è ciò che mostra describe-function`, e i test sulla documentazione lo verificano.
3. Scrivi la funzione con la firma di `LispPrimitive<T>`. Controlla l'arietà con `exact_arity` o `some_arguments`, i tipi restituendo `WrongArgumentType`.
4. Restituisci le liste con `LispExp::proper_list`, mai con `LispExp::form`.
5. Se scorri una lista, usa `expect_list` (o addebita con `ctx.consume_fuel`): altrimenti un ciclo che chiama la primitiva sfugge al limite.
6. Se allochi in base a un numero ricevuto dall'utente, limita il numero.
7. Per un errore con un nome proprio, restituisci `EvalError::Signal`.
8. Registra con `into.function(nome, puntatore, DOC)` nella `install` del modulo e aggiungi i test in `lisp/tests/`.

13 Differenze da Emacs Lisp

rsedit prende in prestito nomi e forme da Emacs Lisp, ma non è Emacs Lisp. La tabella raccoglie le differenze osservate eseguendo l'interprete; chi porta codice da Emacs deve conoscerle.

Argomento	rsedit	Emacs Lisp
Scope	Lessicale per tutte le variabili, anche quelle di <code>defvar</code> .	Dinamico per le variabili speciali (<code>defvar</code>), lessicale altrove.
<code>setq</code> su un nome nuovo dentro una funzione	Crea un legame locale alla chiamata.	Crea una variabile globale.
<code>defun</code> / <code>defvar</code> dentro una funzione	Locali alla chiamata.	Globali.
<code>if</code> con più forme «altrimenti»	Valuta solo la prima.	Le valuta tutte, come un <code>progn</code> .
Valore di <code>while</code>	L'ultima forma eseguita dal corpo.	Sempre <code>nil</code> .
<code>dolist</code> con risultato	La variabile vale ancora l'ultimo elemento.	La variabile vale <code>nil</code> .
Backquote annidato	Un solo livello.	Livelli annidati gestiti.
Numeri	Solo <code>f64</code> ; niente <code>1e20</code> , niente interi separati.	Interi e virgola mobile distinti; notazione esponenziale.
Divisione	<code>(/ 1 3)</code> vale <code>0.333...</code>	Tra interi è intera: <code>(/ 1 3)</code> vale <code>0</code> .
<code>%</code>	Come <code>mod</code> : il segno segue il divisore.	Resto: il segno segue il dividendo.
Caratteri	Nessun tipo carattere né sintassi <code>?a</code> ; <code>make-string</code> prende una stringa e ne usa il primo carattere.	<code>?a</code> è l'intero 97; <code>make-string</code> prende un carattere.
Lambda citata	<code>'(lambda ...)</code> non è chiamabile.	Chiamabile.
Condizioni	Nessuna gerarchia né <code>define-error</code> ; <code>error</code> e <code>t</code> intercettano qualunque errore.	Gerarchie dichiarate.
<code>format</code> con <code>%S</code>	Come <code>%s</code> : <code>(format "%S" "a")</code> vale <code>"a"</code> .	Stampa leggibile: le virgolette restano.
<code>eval-string</code>	Valuta solo la prima espressione.	(non esiste; si usa <code>read + eval</code>)
<code>add-to-list</code>	Accetta più elementi, nessun <code>APPEND</code> ; scrive nell'ambiente del chiamante.	Un elemento, poi <code>APPEND</code> e <code>COMPARE-FN</code> ; aggiorna il simbolo globale.
Espressioni regolari	Sintassi del crate <code>regex</code> di Rust <code>((...))</code> , senza lookaround.	Sintassi Emacs <code>((...))</code> .
Macro	Niente <code>gensym</code> , niente <code>docstring</code> ; espande a ogni valutazione.	<code>gensym</code> , <code>docstring</code> , espansione memorizzata dal compilatore.
<code>yield</code> , <code>fiber</code> , <code>spawn</code> , <code>atomi</code>	Presenti (Sezione 8, Sezione 10).	Assenti.

14 Estendere l'interprete

14.1 Una nuova forma speciale

Una forma speciale serve solo quando gli argomenti non devono essere valutati prima e una macro non basta, per esempio quando la forma deve intercettare errori o sospensioni. Negli altri casi è preferibile una macro in Lisp o una primitiva.

1. Aggiungi un ramo in `eval_special_form_or_call_step`. Il ramo riceve gli argomenti non valutati, l'ambiente, il contesto e il permesso di sospendersi.
2. Decidi la *posizione di coda*: se l'ultima cosa da fare è valutare una forma il cui valore è il risultato, restituisci `TailCall`. Se invece la forma deve vedere un errore che risale, valuta dentro il ramo con `eval`.
3. Decidi la sospendibilità: un corpo che passa da `eval_body_step` o `run_statements` diventa sospendibile senza altro lavoro, perché sono loro a scrivere i frame. Altrimenti un `yield` interno viene rifiutato, che è la scelta sicura.
4. Se intercetti errori, annota `call_frame_depth` prima e chiama `truncate_call_frames` dopo.
5. Se serve un nuovo errore di forma malformata, aggiungilo a `EvalError`; per dargli una condizione con nome, aggiornalo in `error_symbol` e, se porta dati, in `error_data`.
6. Aggiorna questo manuale: la tabella delle forme speciali, quella delle posizioni di coda e, se cambia, quella delle posizioni di `yield`.

14.2 Una nuova variante di LispExp

È la modifica più invasiva. Vanno aggiornati: `PartialEq` e `Debug` in `lispexp.rs`, lo smistamento in `eval_step_permitted`, i predicati e la stampa nelle primitive di base (`lisp_display`, `format`, `eq`), e — se la variante è un contenitore — `form_to_data`, `data_to_form` ed `eval_backquote`. Prima di farlo, conviene chiedersi se un `Atom`, una `Map` o un dato agganciato dall'host non bastino.

14.3 I test

I test stanno in `lisp/tests/` e girano con il contesto `()`.

File	Cosa fissano
<code>eval_tests.rs</code>	le forme speciali e i loro errori di forma
<code>handshake.rs</code>	<code>bootstrap_vm</code> e il limite ai cicli con un host fittizio che misura da sé il carburante; <code>add-to-list</code> e simili
<code>lexical_context.rs</code>	catena degli ambienti, <code>setq</code> , <code>let</code> , scope lessicale, chiusure
<code>nonlocal_exit_tests.rs</code>	<code>catch</code> , <code>condition-case</code> , <code>unwind-protect</code>
<code>yield_tests.rs</code> , <code>fiber_tests.rs</code>	sospensione e ripresa
<code>fuel.rs</code>	scope, ricariche, concessioni
<code>thread_tests.rs</code>	<code>spawn</code> , atomi, carburante per thread
<code>backtrace_tests.rs</code>	il protocollo dei frame
<code>lisp_core_compliance_tests.rs</code>	comportamenti allineati a Emacs Lisp
<code>parser_tests.rs</code> , <code>lexer_tests.rs</code>	lettura e token
<code>base_env_tests.rs</code> , <code>documentation_tests.rs</code> , <code>symbol_tests.rs</code>	primitive di base, docstring, proprietà

15 Appendice

15.1 Glossario

Forma una lista di codice (`LispExp::Form`), letta dal lettore e smistata dal valutatore.

Dato un valore, in particolare una lista di `Cons`.

Primitiva una funzione scritta in Rust e registrata in un `Env`.

Forma speciale un costrutto riconosciuto per nome dal valutatore, i cui argomenti non sono valutati in anticipo.

Istruzione una forma di un corpo il cui valore viene scartato; l'unico punto, insieme alle chiamate in coda, in cui una fiber può sospendersi.

Posizione di coda una forma il cui valore è il valore della forma che la contiene; valutata dal trampolino senza far crescere lo stack.

Trampolino il ciclo di `eval` che esegue le chiamate in coda.

Carburante il numero di passi che uno scope può ancora eseguire.

Frame (di fiber) la descrizione di un blocco sospeso, cioè di che cosa resta da eseguire.

Frame (di backtrace) una voce della pila delle chiamate tenuta dall'host.

Host il programma che incorpora l'interprete e fornisce `T: LispContext`.

15.2 Mantenere aggiornato questo manuale

Questo file è pensato per crescere con il codice. Quando cambi l'interprete, controlla le sezioni che descrivono la parte modificata:

- nuova forma speciale → Sezione 6 (tabelle delle forme e delle posizioni di coda) e Sezione 8 (posizioni di `yield`);
- nuova variante di `EvalError` con condizione propria → Sezione 7;
- nuova primitiva di base o nuovo modulo → Sezione 12;
- nuovo metodo di `LispContext` → Sezione 11;
- nuovo stato per thread → Sezione 10;
- comportamento allineato o reso diverso da Emacs → Sezione 13;
- difetto corretto → aggiorna la descrizione e toglì il rimando al rapporto, poi toglì la scheda da `rapporto-problemi.typ`.

Gli estratti di codice vanno riletti quando cambia la funzione da cui vengono: sono semplificati, ma ogni riga deve corrispondere a una riga del sorgente.

I diagrammi usano gli aiuti di `style.typ`: `canvas`, `node`, `arrow`, `carrow`, `segment`, `group`, `note` e, per le sequenze, `lane`, `msg`, `self-msg` e `seq-note`; le tabelle di riferimento usano `ref-table`. Le coordinate sono in centimetri dall'angolo in alto a sinistra del riquadro; `node` riceve il *centro* della casella, `group` il suo angolo in alto a sinistra. Dopo ogni modifica conviene compilare e guardare le pagine: Typst segnala gli errori di sintassi, non le etichette che si sovrappongono.